



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 58. El procesamiento cooperativo y la arquitectura cliente-servidor. Arquitectura SOA.

Editado por ASTIC en 2019 a partir de los temas numerados anteriormente como:

- **Tema 50. Procesamiento cooperativo y la arquitectura cliente-servidor**
Autores: Leonor Torres / Carmen García / Juan Cantador
Actualización 2004
- **Tema 51. Arquitectura SOA**

ÍNDICE

1.	EL PROCESAMIENTO COOPERATIVO Y LA ARQUITECTURA CLIENTE-SERVIDOR	3
1.1	INTRODUCCIÓN AL PROCESAMIENTO COOPERATIVO.....	3
1.2	SISTEMAS DISTRIBUIDOS	3
1.2.1	Definición	3
1.2.2	Características.....	3
1.2.3	Objetivos	4
1.2.4	Evolución de las Arquitecturas y de los Sistemas Operativos	5
1.2.5	Modelos.....	7
1.2.6	Mecanismos de Comunicación.....	10
1.2.7	Soluciones de Integración de Sistemas Centralizados.....	11
2.	ARQUITECTURAS SOA.....	13
2.1	INTRODUCCIÓN A LAS ARQUITECTURAS SOA	13
2.2	¿QUÉ ES SOA?.....	13
2.2.1	Definición	13
2.2.2	Principios de SOA	14
2.2.3	Beneficios de SOA.....	15
2.3	¿QUÉ ES UN SERVICIO?	16
2.3.1	Definición de Servicio	16
2.3.2	Concepto de orientación a servicios.....	18
2.3.3	Análisis, diseño y desarrollo orientado a servicios	19
2.4	SOA Y WEB SERVICES.....	22
2.4.1	Arquitectura de servicios web.....	22
2.4.2	Web Services como alternativa para implementar arquitecturas SOA.....	23
2.4.3	Estándares y protocolos de Web Services en las arquitecturas SOA	24
2.5	SOA Y PROCESOS DE NEGOCIO.....	27
2.5.1	Procesos de Negocio y BPM.....	27
2.5.2	Organismos de estandarización y estándares	29
2.5.3	BPM y SOA.....	30
2.6	ARQUITECTURA DE REFERENCIA SOA.....	33
2.6.1	¿Qué es una arquitectura de referencia SOA?	33
2.6.2	Contenido de una arquitectura de referencia	35
2.6.3	El concepto de Enterprise Service Bus	38
3.	GLOSARIO	41
4.	TEST	42
4.1	PREGUNTAS DE TEST.....	42
4.2	SOLUCIÓN A LAS PREGUNTAS DE TEST	42
5.	BIBLIOGRAFÍA	43
5.1	BIBLIOGRAFÍA BÁSICA	43
5.2	BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.....	43

1. El procesamiento cooperativo y la arquitectura cliente-servidor

1.1 Introducción al procesamiento cooperativo

El modelo centralizado es el que ha sido ampliamente utilizado en los Sistemas de Información de las grandes organizaciones en décadas anteriores, mediante un Host que ejecutaba el 100% de la lógica del sistema, residiendo únicamente en el terminal de usuario las funciones de presentación.

La principal ventaja de este modelo es que se dispone y se procesa toda la información en la misma máquina física, con lo que el software del sistema es mucho más sencillo y fácil de gestionar. De las desventajas se puede nombrar su poca capacidad de crecimiento o escalabilidad, ya que, aunque los equipos que dan soporte a los mismos son de una capacidad extrema, una vez se llega al límite de utilización del mismo, la única posibilidad de crecimiento es la adquisición y sustitución por un nuevo equipo con más potencia y capacidad, con el agravante de que estos sistemas son de los más caros del mercado. Otro de los problemas de este modelo es su poca capacidad de integración con otros sistemas de información. Son prácticamente islas en los que la única posibilidad de intercambio de información se hace por medio de procesos diferidos *offline*. Otro defecto de estos sistemas son los relativos a la disponibilidad de los mismos, derivada principalmente de la tolerancia a fallos que proporcionan ya que, a pesar de que los sistemas mainframe son de los más robustos que se conocen, cuando se produce un error en el sistema, el sistema en su totalidad falla, haciendo que la disponibilidad en estos casos no sea la deseada.

El **procesamiento cooperativo** es aquel en que dos o más elementos lógicos diferentes interactúan entre sí para la realización de una tarea común. Surgió para intentar dar respuesta a los problemas y debilidades del modelo centralizado, aunque este modelo tiene sus propios problemas derivados por su propia naturaleza distribuida.

Es muy sencillo encontrar ejemplos de sistemas que implementan el procesamiento cooperativo, ya que, en la actualidad, prácticamente todos los sistemas lo desarrollan. Desde varios sistemas centralizados que trabajan de forma coordinada por medio de soluciones de integración, pasando por la propia arquitectura cliente/servidor de la que el mundo Web e Internet es su mejor ejemplo, hasta los nuevos sistemas de computación Grid, en los que el procesamiento cooperativo es llevado a su máxima expresión.

1.2 Sistemas Distribuidos

1.2.1 Definición

Los sistemas que en su funcionamiento se basan en el procesamiento cooperativo se denominan sistemas distribuidos. En la literatura existen bastantes definiciones acerca de los mismos, por ejemplo, Sloman & Kramer en 1987 lo hacen de la siguiente forma:

“Un **sistema distribuido** es aquel en el cual varios procesadores autónomos y repositorios de datos que soportan procesos y/o bases de datos, interactúan con el fin de cooperar para lograr un **objetivo global**. Los procesos coordinan sus actividades e intercambian información por medio de la transferencia de información a través de una red de comunicación”.

De esta definición se pueden sacar distintas conclusiones, como, por ejemplo, la idea de la utilización de un conjunto de recursos distribuidos para la realización de un trabajo común. Esta idea se ve refrendada por otra definición, en este caso de Tanenbaum en 1995:

“Un **sistema distribuido** es una colección de computadoras independientes que aparece ante los usuarios del sistema como una única computadora”

La definición de Tanenbaum establece la idea de sistema distribuido como la forma de ofrecer un **servicio único** a los usuarios a partir del uso de un conjunto de computadoras totalmente independientes.

1.2.2 Características

En los sistemas centralizados existe un reloj común que sirve para sincronizar y coordinar cualquier proceso o comunicación de mensajes. En los sistemas distribuidos, al ser la suma de varios sistemas

totalmente independientes, cada uno con sus relojes de sincronización independientes, a la hora de la coordinación y paso de mensajes no existe un reloj común, por lo que se tienen que buscar formas alternativas de sincronización. La **no existencia de un reloj común** es más problemática dependiendo del tipo de sistema distribuido, ya que, por ejemplo, si se utiliza el modelo cliente/servidor, las soluciones de sincronización son relativamente sencillas, pero sin embargo en sistemas del tipo SSI Single System Image (Imagen única del sistema), en los que el conjunto de sistemas se ve como uno solo, y en los que los procesos de usuario se pueden ejecutar indistintamente en distintas máquinas, la sincronización es algo más compleja. Además de los problemas de sincronización, por la no existencia de reloj común, se tienen que resolver los llamados problemas de orden o causalidad: una vez ya se han ejecutado distintos trabajos en distintos nodos del sistema, es bastante complejo averiguar en qué orden se han ejecutado estos.

La **conurrencia global** es la capacidad de que la ejecución de los distintos elementos del sistema se realice de forma paralela. Esta es otra característica típica de los sistemas distribuidos. La gran ventaja que ofrecen es que se pueden paralelizar trabajos entre los distintos elementos, aunque en los casos en los que tienen que acceder a un recurso compartido, p.ej. la escritura en un mismo fichero físico, y para evitar bloqueos y estados incongruentes del sistema, se tienen que utilizar algoritmos de 'exclusión mutua' para evitar que no accedan en el mismo momento.

Una característica que, bien gestionada, supone una gran ventaja en este tipo de sistemas es que los **fallos son independientes** del nodo del sistema en el que se producen, por lo que en el caso de que un nodo fallara los demás podrían continuar ejecutando sus acciones. Esto permite la consecución de los trabajos con mayor efectividad, ya que el sistema en su conjunto continúa trabajando. Eso sí, cuando se produce un fallo en uno de los nodos del sistema, el fallo se tiene que gestionar de forma adecuada, ya que, aunque el resto del sistema continúe funcionando, el nodo problemático puede dar lugar a resultados erróneos. Así que, cuando se produce un fallo en un nodo, generalmente se tiene que aislar del resto sistema mientras se recupera del mismo.

Otra característica importante en los sistemas distribuidos es la **heterogeneidad**: tanto las arquitecturas de los nodos que componen el sistema distribuido, como los sistemas operativos de los nodos, no tienen por qué ser iguales. Esta flexibilidad es importante a la hora de ahorrar en costes: aunque se tenga un nodo de un determinado fabricante, este no tiene por qué determinar la arquitectura del resto del sistema.

1.2.3 Objetivos

Los objetivos principales que buscan los sistemas distribuidos son los siguientes:

- Transparencia
- Fiabilidad (disponibilidad y coherencia)
- Rendimiento
- Escalabilidad
- Flexibilidad
- Seguridad

Transparencia

La transparencia se consigue cuando se consigue que, desde el punto de vista del usuario, el sistema se comporte como si fuera un sistema centralizado:

- El acceso a un recurso remoto deberá ser igual que si se accediera a un recurso local.
- Se deberá poder acceder a los distintos recursos sin conocer la localización de los mismos. Para acceder a un recurso remoto no habrá que conocer de qué nodo depende.
- Los diferentes recursos (p.ej. ficheros) podrán migrar de localización sin afectar a los usuarios.
- El acceso concurrente a un mismo recurso no afectará a los usuarios.
- La existencia de réplicas de los recursos no afectará a los usuarios.
- La ocurrencia de fallos en alguno de los nodos no afectará a los usuarios.
- El crecimiento del sistema no afectará a los usuarios.
- El posible carácter heterogéneo de los nodos del sistema no afectará a los usuarios.

Fiabilidad

La fiabilidad en los sistemas distribuidos se tiene que buscar desde dos puntos de vista distintos:

- Fiabilidad como disponibilidad: se busca un sistema de alta disponibilidad mediante la redundancia de nodos y recursos.
- Fiabilidad como coherencia: se tiene que buscar que la información que procesa el sistema siempre sea coherente, aspecto que en sistemas en los que se utiliza la redundancia se dificulta bastante.

Rendimiento

El rendimiento que se persigue no debe ser peor que en un sistema centralizado y debe ser proporcional al número de procesadores empleado. Para conseguirlo se deben tener unas buenas políticas de equilibrado de carga.

En este aspecto el principal problema es que, a más número de procesadores, más elementos críticos corren el riesgo de convertirse en cuellos de botella, por ejemplo, la red de comunicaciones.

Escalabilidad

El diseño del sistema tiene que tratar de evitar, principalmente en sistemas que vayan a contar con un gran número de elementos de proceso, los cuellos de botella (p.ej.: componentes centralizados, tablas centralizadas, algoritmos centralizados).

Si se diseña de forma cuidadosa y planificada, que el sistema crezca mediante la adición de nuevos nodos al sistema, nos proporcionará un aumento del rendimiento proporcional al número de procesadores que añadamos.

Flexibilidad

La flexibilidad se entiende como la capacidad de ampliar o extender el sistema con nuevas funcionalidades de forma sencilla.

Un ejemplo claro de la flexibilidad es la que se consigue con el uso de soluciones de sistemas abiertos, ya que, al estar basados en estándares, interfaces y protocolos públicos, no se depende de ningún fabricante a la hora de extender las funcionalidades del mismo.

1.2.4 Evolución de las Arquitecturas y de los Sistemas Operativos

La evolución de los sistemas y de las arquitecturas hardware han ido de la mano de la evolución de los sistemas operativos que les dan los servicios básicos.

Con el estudio los sistemas distribuidos, el conocimiento de la evolución de los sistemas operativos y de sus distintos tipos es indispensable, ya que son estos los que tienen que dar las capacidades y servicios básicos para poder implementarlos. De hecho, la evolución de los sistemas operativos ha propiciado la evolución de los sistemas distribuidos y viceversa.

1.2.4.1 Tipos de SO

S.O. Centralizados

Los sistemas operativos centralizados fueron el punto de partida para el desarrollo de los primeros sistemas operativos que permitieron la implementación de los primeros sistemas distribuidos.

S.O. en Red

Los sistemas operativos de red son sistemas operativos convencionales que disponen de servicios de red. Las características que más los definen son las siguientes:

- No dan la visión de una única máquina, sino de máquinas independientes.
- Cada máquina ejecuta una copia del S.O.
- Proveen de protocolos de comunicación para el intercambio de recursos y acceso a servicios de alto nivel.

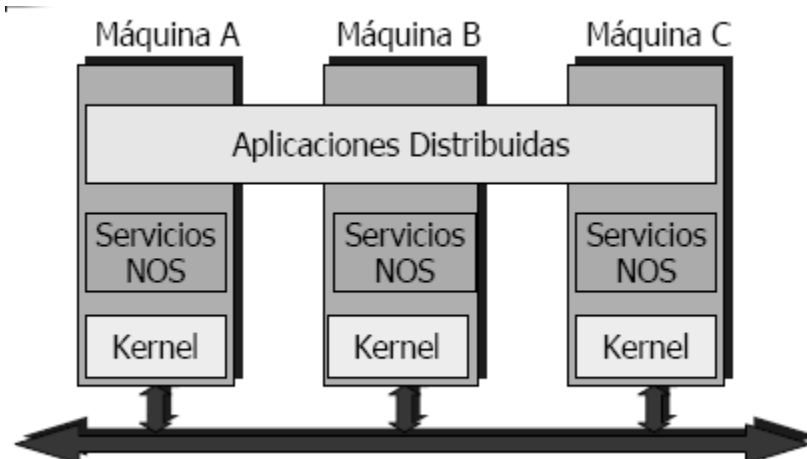


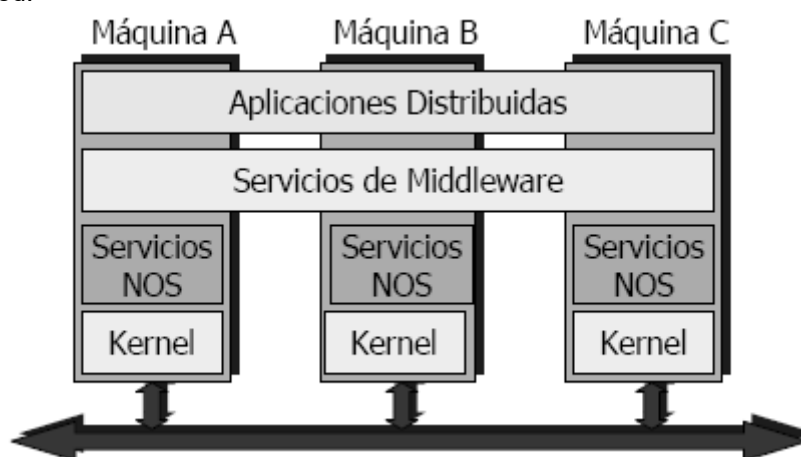
Ilustración 1. Sistema operativo en red

S.O. Cooperativos o basados en Middlewares

Son los sistemas orientados a servicios de alto nivel que requieren el soporte de mecanismos de comunicación en base a los cuales los protocolos de comunicaciones de alto nivel se construyen.

Estos mantienen el grado de transparencia sacrificando la visión de único sistema. Al igual que los SO de Red, siguen manteniendo la visión de sistemas autónomos independientes.

Se suelen construir en base a *middlewares* (CORBA, DCE, DCOM...). Los sistemas distribuidos que implementan resultan de la integración de múltiples servicios proporcionados por diferentes elementos de la red.

Ilustración 2. Sistemas operativos basados en *middlewares*

S.O. Distribuidos

A pesar de su nombre, no son la solución más extendida para implementar sistemas distribuidos, pero sí son los que llevan los mismos a su máxima expresión.

Un **sistema operativo distribuido** es un conjunto de procesadores interconectados por redes que ocultan dicha característica mostrando una visión al usuario de "*uniprocador virtual*".

Los sistemas operativos distribuidos se ejecutan sobre un conjunto de nodos independientes distribuidos e interconectados por medio de redes, haciendo creer a los usuarios que se trata de un sistema centralizado. Su complejidad hace que este tipo de sistemas operativos alcancen el objetivo de dar la visión de una única máquina hasta cierto punto.

La principal aportación de este tipo de sistemas operativos es que han desarrollado nuevos conceptos y planteamientos que se han podido trasladar a otros modelos de sistemas distribuidos.

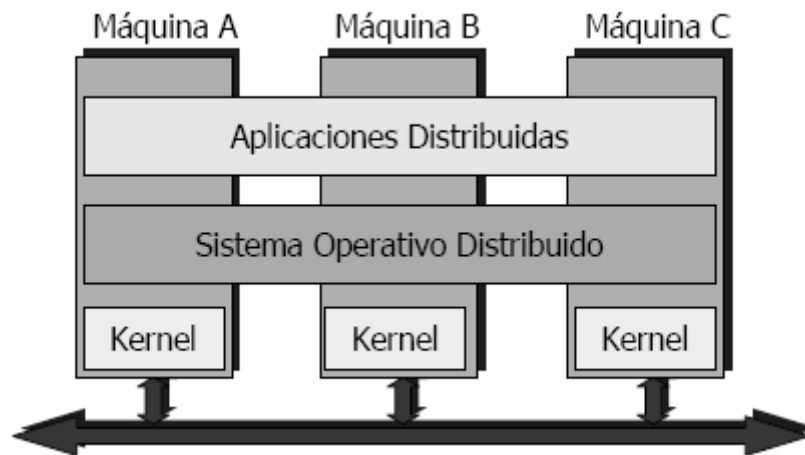


Ilustración 3. Sistema operativo distribuido

Ejemplos de sistemas operativos distribuidos: Mosix/Open Mosix, OpenSSI

1.2.4.2 Evolución de los sistemas operativos distribuidos

Primeros sistemas operativos de red

A primeros de los 80 se empezaron a incluir en los SO convencionales servicios de red, p.ej. el UNIX 4BSD de 1980.

Paulatina incorporación de más funcionalidad

A mediados de la misma década estos servicios de red habían evolucionado hasta proveer servicios tipo NFS (Network File System), RPC (Remote Procedure Call), NIS (Network Information Service), p.ej. Open Network Computing RPC de Sun (1985).

Primeros sistemas operativos distribuidos

A finales de la década comenzaron a surgir los primeros sistemas operativos distribuidos, pero basados en arquitecturas convencionales (monolíticas), p.ej. Sprite de la Universidad de Berkeley (1988).

Más adelante comenzaron a surgir otros sistemas operativos distribuidos basados en microkernel, donde el núcleo del sistema operativo, a diferencia de los monolíticos, se aligera y ofrece sólo servicios básicos para el funcionamiento del mismo, p.ej. Chorus de INRIA.

1.2.5 Modelos

Los modelos más habituales de Sistemas Distribuidos son los siguientes:

1.2.5.1 Modelo Cliente/Servidor

Es el modelo más simple y extendido de sistemas operativos distribuidos. Se basa en la existencia de un sistema denominado **Servidor** que proporciona un servicio a través de un interfaz o protocolo definido, y de un sistema denominado **Cliente** que es quién lo solicita:

- Cliente: solicita el servicio.
- Servidor: proporciona el servicio.

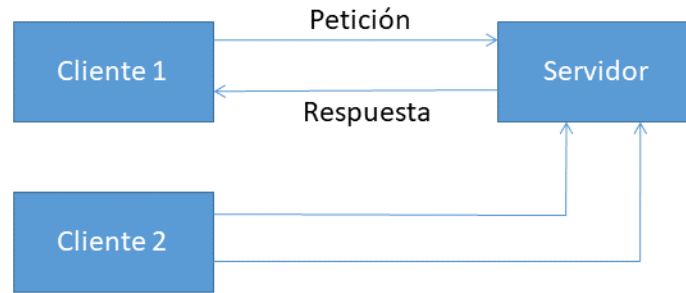


Ilustración 4. El modelo cliente/servidor

Categorías de Servidores

A continuación, se presenta una lista de los servidores más comunes:

- **Servidores de archivos:** servicio que se encarga de proporcionar archivos para clientes. El cliente solicita los archivos y el servidor los ubica y se los envía (p.ej. NFS Network File System desarrollado originalmente por Sun).
- **Servidores de Base de Datos:** son los servidores que almacenan gran cantidad de datos e información de forma estructurada y permiten su definición, alta, modificación y consulta. (p.ej. Oracle, SQL Server, MySQL, etc.).
- **Servidores Web:** son los servicios que proporcionan el acceso a páginas y aplicaciones web. El cliente desde un navegador web (Internet Explorer, Mozilla Firefox) realiza una petición escribiendo una URL y el servidor recibe la misma y envía la página correspondiente a esa URL.
- **Servidores de correo:** gestionan el envío y recepción de correo de un grupo de usuarios.
- **Otros servidores:** de impresión, servidores de aplicaciones, etc.

La evolución del modelo Cliente/Servidor ha generado las siguientes arquitecturas:

- **Arquitectura cliente/servidor de 2 capas:** el servidor divide el servicio en dos capas, la primera de presentación y lógica de negocio, y la segunda de acceso a datos. Hasta hace relativamente poco tiempo, todas las aplicaciones web se desarrollaban en base a esta arquitectura. Al estar poco estructuradas, el mantenimiento de las mismas era costoso, debido a que se mezclaba la presentación de las aplicaciones con la lógica de negocio.
- **Arquitectura cliente/servidor de 3 ó N capas:** el servidor divide el servicio en 3 o más capas. Generalmente una capa de presentación, otra de lógica de negocio, y la tercera y última de acceso a datos. La mayoría de las aplicaciones web actuales se desarrollan en base a esta arquitectura.

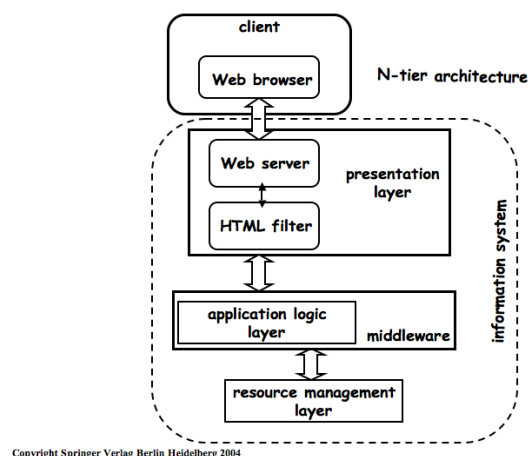


Ilustración 5. Arquitectura de N capas

1.2.5.2 Modelo Proxy/Cache

Otro modelo de sistemas distribuidos, evolucionado directamente del modelo cliente/servidor es el modelo Proxy. Se basa en la existencia de tres roles diferentes en la interacción:

- **Cliente:** solicita el servicio.
- **Proxy:** intermediario (a ojos del cliente es como si fuera el servidor). Si el proxy tiene memoria de las respuestas para las peticiones se denomina **caché**.
- **Servidor:** proporciona el servicio.

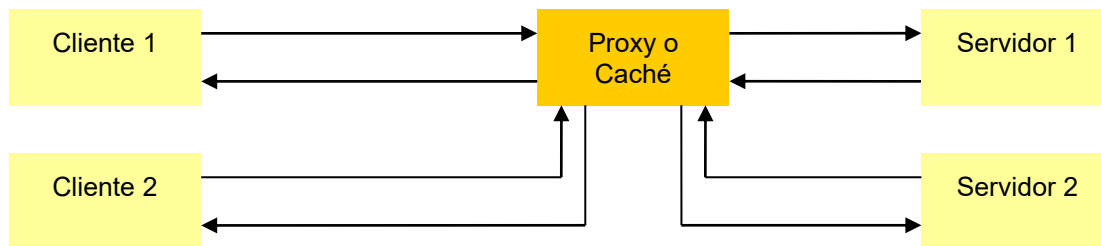


Ilustración 6. El modelo Proxy/Cache

El propósito de los servidores proxy es incrementar la disponibilidad y las prestaciones del modelo cliente-servidor, reduciendo la carga en las redes de comunicaciones e incluso ofreciendo otros valores añadidos como, por ejemplo, la posibilidad de centralizar las peticiones de varios servicios en un único proxy.

1.2.5.3 Modelo Peer to Peer

En el modelo peer to peer o punto a punto, las entidades que participan lo hacen 'de igual a igual' a través de un protocolo de diálogo con primitivas de interacción.

Todos los nodos desempeñan tareas semejantes, interactuando cooperativamente como iguales para realizar una actividad distribuida o cómputo sin distinción entre clientes y servidores.

Los nodos pares mantienen la consistencia de los recursos y sincronizan las acciones a nivel de aplicación.

Ejemplos de este tipo de modelo son los sistemas de computación Grid o las tan extendidas redes P2P de compartición y descarga de ficheros en Internet.

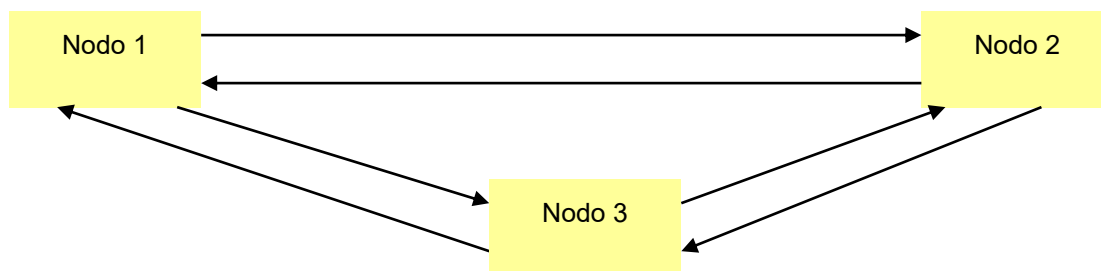


Ilustración 7. El modelo Peer to peer

1.2.5.4 Otros Modelos

Existen otros tipos de modelos menos importantes, pero no por ellos menos extendidos:

Código móvil: código que se traslada de un servidor a un cliente y que se ejecuta en el cliente (p.ej. applets o componentes Active X).

Agente móvil: es un programa que se traslada en la red, de un computador a otro, realizando una tarea para alguien (p.ej. recolectar información).

Clientes ligeros: en el cliente sólo se ejecuta una interfaz basada en ventanas, mientras que la aplicación se ejecuta en un servidor remoto, usualmente muy potente (multiprocesador, clústeres, etc.).

1.2.6 Mecanismos de Comunicación

Para que un sistema distribuido se pueda llevar a cabo, una de las partes imprescindible es la red de comunicaciones. Los distintos mecanismos que se suelen utilizar para implementar sistemas distribuidos son los siguientes:

1.2.6.1 Paso de Mensajes. Primitivas *send* y *receive*

Se basa en el paso de mensajes mediante las primitivas *send* y *receive*. Cada pareja send-receive transmite un mensaje entre cliente y servidor, por lo general de forma asíncrona. Habitualmente:

- **Send** es no bloqueante (el nodo envía el mensaje y no espera la respuesta del receptor).
- **Receive** es bloqueante (el nodo que espera un mensaje no realiza ninguna acción hasta que le llega).

1.2.6.2 Llamadas a procedimientos remotos (RPCs)

El mecanismo de comunicación mediante llamadas a procedimientos remotos, más conocidos como RPCs, fue un mecanismo de comunicación propuesto inicialmente por Birrel y Nelson en 1985. Su objetivo principal es que la comunicación entre sistemas se realice de una forma más cómoda y a más alto nivel, de forma que, en lugar de enviarse mensajes a través de primitivas, lo que se realiza es la llamada desde un cliente a una función o procedimiento de un servidor.

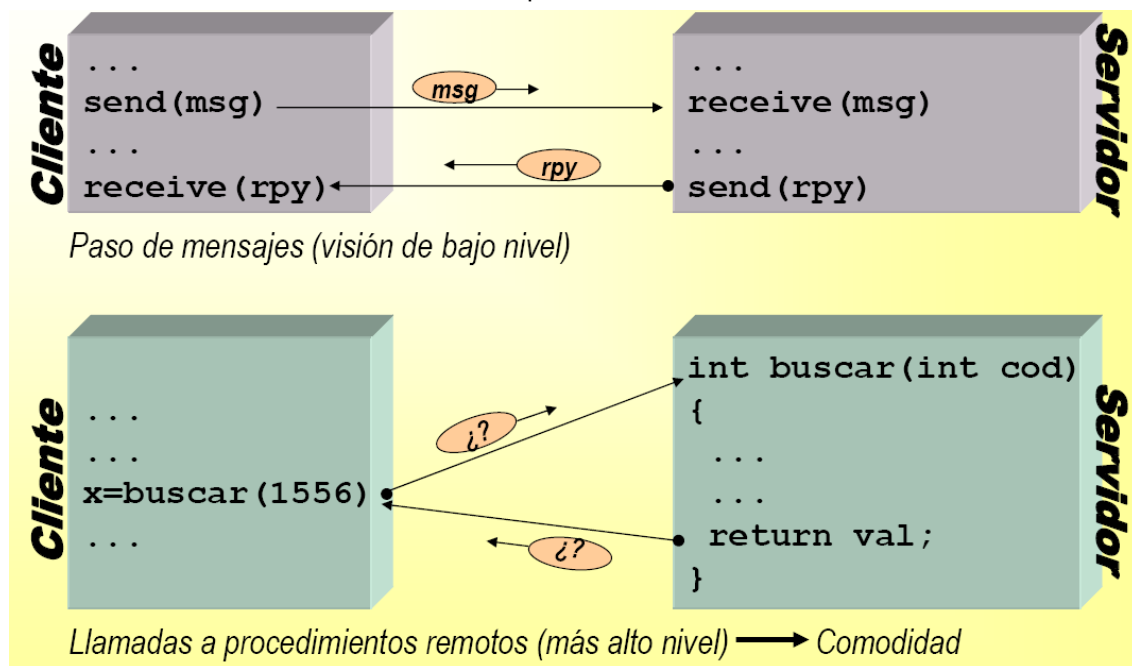


Ilustración 8. Mecanismos de comunicación

Su funcionamiento es el siguiente:

Cliente:

- El proceso realiza una llamada a una función (p.ej. sumar (3,4)).
- Dicha llamada empaqueta los argumentos en un mensaje y se los envía a otro proceso.
- Queda a la espera del resultado.

Servidor:

- Se recibe un mensaje consistente en varios argumentos.
- Los argumentos son usados para llamar a una función en el servidor.
- El resultado de la función (3+4=7) se empaqueta en un mensaje que se retransmite al cliente.

Objetos Remotos

Los objetos remotos son la evolución natural de las llamadas a procedimientos remotos con orientación a objetos. En lugar de realizar desde el cliente una llamada a una función de un servidor,

lo que se realiza desde el cliente es instanciar un objeto del servidor y poder llamar a los métodos del mismo como si ese objeto fuera local.

SSI: Single System Image

Una imagen única de sistema (Single System Image, SSI) es una propiedad de un sistema que oculta la naturaleza heterogénea y distribuida de los recursos, y los presenta a los usuarios y a las aplicaciones como un recurso computacional unificado y sencillo. SSI significa que el usuario tiene una visión global de los recursos disponibles independientemente del nodo al que están físicamente asociados esos recursos.

Realmente, los sistemas que se desarrollen bajo el paradigma de Single System Image no tienen que utilizar mecanismos de comunicación distribuidos de forma explícita, ya que estos vendrían dados de forma implícita por el mismo sistema operativo. Los sistemas operativos que proveen SSI son los sistemas operativos distribuidos (ver S.O. Distribuidos en el apartado 1.2.4.1 Tipos de SO.)

1.2.7 Soluciones de Integración de Sistemas Centralizados

Una forma de reaprovechar la potencia de los sistemas ya desarrollados orientados a un modelo centralizado, generalmente bajo mainframe, es integrarlos con el resto de sistemas de la organización. De esta forma, se obtiene como resultado un sistema nuevo o conjunto de sistemas que funcionan bajo el paradigma de los sistemas distribuidos, ya que serían un conjunto de sistemas interactuando de forma coordinada para conseguir un fin común.

La mayoría de las soluciones de Integración se basan en la utilización de ciertos middlewares, que, aunque no sean estrictamente necesarios, facilitan la integración y en los casos en que se integran más de dos sistemas, reducen la dependencia entre sistemas, ya que la dependencia será sólo de cada sistema con el middleware.

1.2.7.1 Tipos de Middlewares

Repasando la historia del middleware desde los años 80 se pueden distinguir tres categorías de productos: estaciones de mensajería, motores de integración y buses de integración.

1.2.7.1.1 Estaciones de mensajería

Las estaciones de mensajería (EM) proporcionan asistencia a los sistemas en lo referente a la gestión del medio de comunicación. Concretamente liberan a las aplicaciones de la servidumbre que supone mantener los parámetros de conexión de todos sus interlocutores y de gestionar directamente con ellos el envío y recepción de mensajes.

Con una EM lo único que precisan saber las aplicaciones es cómo intercambiar mensajes con ella. La EM asume el compromiso de hacer llegar los mensajes a su destino de una manera rápida y segura que se puede especificar mediante parámetros de calidad y acuerdos de nivel de servicio. Estos acuerdos se expresan en términos de un tiempo máximo de entrega, el reintento un número determinado de veces o durante cierto tiempo si el sistema remoto no está disponible, etc.

1.2.7.1.2 Motores de integración

Los motores o brókeres de integración aparecieron como una evolución de las EM hacia finales de los 90. Los motores de integración heredan todas las características de las EM a las que añaden un nuevo tipo de funcionalidades basadas en la interpretación y tratamiento de los mensajes (recuérdese que las EM no intervienen en el contenido de los mensajes). Entre los nuevos servicios cabe destacar la traducción de mensajes y el enrutamiento basado en el contenido. La traducción de mensajes es muy interesante porque en la práctica poner de acuerdo todos los sistemas participantes para que usen el mismo lenguaje (o incluso la misma versión) puede ser un reto considerable, o directamente imposible. A medida que aumenta el nivel del proyecto de integración (departamental, local, corporativo, regional, sectorial...), menor es el control sobre los sistemas participantes y mayor la probabilidad de que el escenario sea multilingüe. Esta característica se suele ver agravada por el hecho de que la mayoría de los sistemas sólo son competentes en el manejo de un lenguaje. Los motores de integración ayudan a poner orden en esta torre de Babel haciendo que las barreras lingüísticas no sean un obstáculo para las interacciones.

Procesos de negocio

Los motores de integración fueron evolucionando, llevándose la lógica de integración de las aplicaciones (y no solamente el control del medio de comunicación) a la capa de middleware.

1.2.7.1.3 Buses de integración

El concepto de bus de integración empresarial (o ESB, por Enterprise Service Bus) apareció en escena en 2002. Es un tipo de middleware diseñado para superar las limitaciones de los motores de integración y responder a las necesidades de integración de las organizaciones a una escala superior a la considerada hasta entonces. La medida de esa nueva escala la proporciona el concepto de empresa u **organización extendida**, es decir aquella con varias sedes, tal vez repartidas por distintos continentes, que establecen procesos de negocio con otras organizaciones sobre cuyos sistemas de información no tiene control (Compárese con los motores de integración, diseñados para resolver los problemas de integración dentro de una LAN). El escenario no es tan ajeno como podría parecer. Una Consejería de Sanidad, con la red de hospitales públicos, centros de atención primaria y centros privados es un buen ejemplo de organización extendida.

1.2.7.1.4 Soluciones

Existen gran número de soluciones de middleware disponibles en el mercado. El dominador en este segmento actualmente es IBM, con IBM Websphere MQ (antes MQSeries), aunque también son muy conocidas y son muy usadas, entre otras, las soluciones Bea Tuxedo, Entire X y Oracle Fusion Middleware.

2. Arquitecturas SOA

2.1 Introducción a las Arquitecturas SOA

Las arquitecturas orientadas a servicios, comúnmente conocidas por su acrónimo anglosajón SOA (*Service Oriented Architectures*), han permitido dotar de flexibilidad y conseguir un gran ahorro de costes en las cuantiosas inversiones realizadas en Tecnologías de la Información y las Comunicaciones.

Su virtud reside en que **facilitan la reutilización de componentes software y procesos de negocio** independientemente de la plataforma tecnológica en que estén desarrollados, que en muchos casos se trata de componentes ya existentes en el abanico de soluciones tecnológicas de las organizaciones.

Sin embargo, como otras muchas tecnologías, requiere de un **proceso de aprendizaje** para aprovechar adecuadamente sus capacidades y de aprender a separar “*el trigo de la paja*” dentro del **mercado de soluciones**.

Los retos principales que debe afrontar la tecnología SOA comprenden:

- Todos los principales proveedores del mercado afirman haber adoptado el paradigma SOA y publican y promueven su propia visión y referencia. Sin embargo, muchos grandes proveedores en realidad lo que han realizado es absorber a pequeñas y volátiles empresas con iniciativas SOA exitosas.
- Las arquitecturas SOA vinculan el “mundo TIC” con la “visión de negocio”, dos perspectivas que no se comunican de forma efectiva en muchas ocasiones y que, a menudo, tienen objetivos y motivaciones contrapuestas (coste reducido, énfasis en calidad, mayor agilidad, mantenibilidad, etc.).
- Existen diversos cuerpos de estandarización y organismos detrás de la definición y las líneas guía de las arquitecturas SOA, lo que añade una mayor confusión, problemas de interoperabilidad, y adopción más lenta de las mismas.
- La recurrente falta de un vocabulario común e interpretaciones incorrectas sobre lo que las arquitecturas SOA significan. Como veremos más adelante, es corriente asociar unívocamente los *Web Services* con las arquitecturas SOA, fuente común de confusión.

2.2 ¿Qué es SOA?

2.2.1 Definición

Podemos afirmar que no existe una definición única de lo que es SOA o que existen múltiples definiciones según sea el organismo de estandarización, empresa o consultora del sector TIC que la emita.

No obstante, podemos extraer una sencilla definición por la propia traducción del término anglosajón SOA como:

“Un paradigma de arquitectura software que cuenta con la orientación a servicios como su principio fundamental de diseño”

El elemento a destacar es la característica de “*orientación a servicios*”, que es la que conforma una arquitectura que utiliza servicios débilmente acoplados para cubrir los requisitos de procesos de negocio y requisitos de usuario, como más adelante veremos.

Más allá de la existencia de varias definiciones, es interesante indicar que, en la práctica, **SOA significa diferentes cosas para diferentes personas**, según el perfil profesional que se esté ejerciendo en el contexto de una organización.

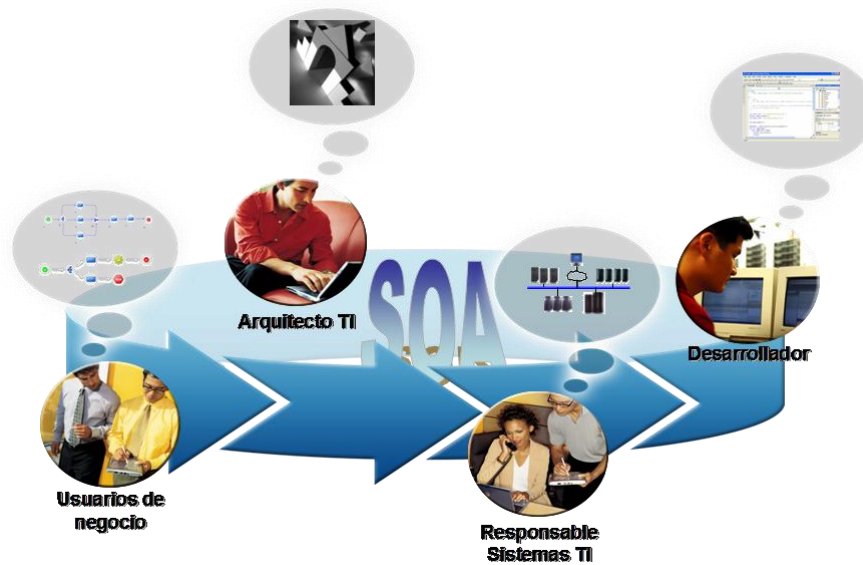


Ilustración 9. Visión de SOA según el perfil profesional de la persona

Así, para un *arquitecto de desarrollo de proyectos TIC*, SOA significa la **definición de la arquitectura corporativa** y los procesos que permiten a los Sistemas de Información desarrollar y desplegar funcionalidades de negocio rápidamente.

Para los *responsables de la explotación de los Sistemas de Información* de una línea de actividad, SOA aporta la **governabilidad**, la **organización** y el **proceso para la gestión de aplicaciones**, así como “*bloques de negocio constituyentes*”, para su reutilización en otros proyectos con el fin de reducir costes.

La explicación de esta disparidad de perspectivas o percepciones es sencilla: los recursos de una red en un entorno SOA se ponen a disposición como servicios independientes que pueden ser accedidos sin el conocimiento de la implementación de la plataforma subyacente, de forma que estos conceptos pueden aplicarse a sistemas software, sistemas de negocio y otros muchos tipos de sistemas informáticos productores/consumidores.

A otro *nivel de carácter más estratégico*, las arquitecturas SOA se han convertido en una **estrategia de negocio** para **mejorar los flujos de información** y alcanzar los objetivos de la organización, en términos de incremento de los ingresos, mejorar la satisfacción del cliente, mejora la calidad de los productos, potenciar la agilidad, etc.

La relación entre los flujos de negocio y las arquitecturas SOA es uno de los aspectos que más ha influido en su aceptación, y que veremos más adelante al comentar la relación entre BPM y SOA.

2.2.2 Principios de SOA

Es comúnmente aceptado que los siguientes **principios guía** definen las reglas básicas para el desarrollo, mantenimiento y uso de las arquitecturas SOA:

- **Reutilización**
- **Interoperabilidad**
- **Granularidad**
- **Modularidad**
- **Composición**

- *Componentización*
- *Conformidad con estándares* (tanto de ámbito común como específicos de la industria).
- Identificación de servicios, *categorización, provisión y entrega, y monitorización y traza*.

Los siguientes **principios arquitectónicos de diseño** se derivan de la definición y diseño de servicios, y son intrínsecamente aplicables a las arquitecturas SOA por su carácter de “**orientación a servicios**”:

- **Encapsulación:** los servicios ocultan el código y la forma en la que implementan las funcionalidades que ofrecen hacia fuera, exponiendo una interfaz con dicha funcionalidad accesible.
- **Débil acoplamiento:** los servicios mantienen una relación entre sí que minimiza las dependencias entre ambos.
- **Contrato:** los servicios se adscriben a un acuerdo de comunicaciones, definido colectivamente por uno o más documentos de descripción del servicio.
- **Abstracción:** al margen de lo descrito en el contrato de servicio, los servicios ocultan la lógica al exterior.
- **Reutilización:** la lógica se divide en servicios con la intención de promover la reutilización.
- **Composición:** un conjunto de servicios puede ensamblarse y coordinarse para conformar servicios compuestos.
- **Autonomía:** los servicios tienen control sobre la lógica que encapsulan.
- **Optimización:** en igualdad de circunstancias, los servicios de mayor calidad se consideran preferibles sobre los de baja calidad.
- **Descubrimiento:** los servicios se diseñan para ser descriptivos “hacia afuera” de forma que puedan ser localizados y accedidos a través de mecanismos de descubrimiento.

Finalmente, a la hora de considerar la implementación de una arquitectura SOA, deberán tenerse en cuenta los siguientes aspectos:

- **Arquitectura de referencia SOA**, que proporcione diagramas detallados de la arquitectura, requisitos detallados, patrones de diseño, opciones tecnológicas y opiniones sobre los estándares utilizados, etc.
- **Gestión del Ciclo de Vida en SOA**, que describe el ciclo de vida completo de los servicios y proporciona un proceso detallado de gestión de los mismos desde su creación a su retirada/eliminación o su reposición por nuevas versiones.
- **Grado y Modelo de Madurez**, cuyo objetivo es acelerar la adopción y disminuir el riesgo de implantar arquitecturas SOA en una empresa u organización, modelando para ello el nivel de madurez de la arquitectura de Tecnologías de la Información de la empresa u organización.

2.2.3 Beneficios de SOA

De forma general, cualquier empresa u organismo público debe enfrentarse con el problema de gestionar dos asuntos importantes y, a veces, contrapuestos: la habilidad de adaptarse a los cambios rápidamente y con agilidad, junto a la necesidad de reducir costes.

En el contexto empresarial, el cambio es esencial para adaptarse a factores internos como adquisiciones o reestructuraciones, o externos como fuerzas competitivas o requisitos de cliente. Para la Administración, la adaptación de las políticas públicas a los cambios que se producen en la sociedad y en las demandas de los ciudadanos, junto a los cambios organizativos y de gestión, también presentes, requieren de forma parecida de las habilidades mencionadas.

Para ello, la infraestructura de los Sistemas de Información que dan el soporte de las actividades de las organizaciones debe ser eficiente en costes, y es ahí donde las arquitecturas SOA ofrecen varios beneficios a considerar en el dinámico entorno en el que nos movemos.

Los principales beneficios que ofrece SOA son:

- Permite **potenciar los activos preexistentes**

Las arquitecturas SOA proporcionan un nivel de abstracción que permite a una organización potenciar la inversión realizada en Tecnologías de la Información y las Comunicaciones, ofreciendo sus activos como servicios que proporcionan funcionalidades de negocio. De esta forma, las organizaciones pueden potencialmente continuar obteniendo valor de los recursos existentes.

- **Facilita la integración**

El punto de integración en las arquitecturas SOA es la especificación del servicio, no su implementación. De esta forma, se proporciona transparencia de la implementación y de la tecnología subyacente, y se minimiza el impacto de los cambios. La integración se simplifica y la complejidad se gestiona mejor, aislando los posibles problemas de la cadena de valor.

- Mayor **capacidad de respuesta**

La habilidad de componer nuevos servicios a partir de servicios existentes proporciona una ventaja significativa que permite a una organización ser más ágil a la hora de responder a las demandas de las necesidades de negocio y de sus clientes.

- **Reducir costes** e incrementar la **reutilización**

Al disponer de los servicios básicos de negocio expuestos de forma que se promueve el débil acoplamiento, es posible la combinación y utilización de los mismos para cubrir las diferentes necesidades de negocio que van surgiendo, lo que supone a su vez menor duplicación de recursos, mayor potencial de reutilización y menor coste.

Es importante apuntar, no obstante, que las arquitecturas SOA no son el “*santo grial*” que solventa todos los problemas, y además la migración a una arquitectura SOA es una tarea muy compleja y comprometida, no exenta de riesgos.

Es preferible, por ello, migrar un conjunto parcial de funcionalidades de negocio de carácter sectorial en lugar de toda la infraestructura TI de la organización, y, de forma paulatina, ir incrementando dichas funcionalidades conforme surgen nuevas necesidades de negocio.

2.3 ¿Qué es un Servicio?

2.3.1 Definición de Servicio

De forma general, podemos definir un **servicio** como:

“Un recurso que representa una capacidad de realizar tareas y una funcionalidad coherente desde el punto de vista de las entidades proveedoras y solicitantes del mismo”

En términos más informáticos, podemos decir que existe una lógica, una pieza de software, que hace algo de utilidad y que es invocada por otra pieza de software que se constituye en el consumidor.

Para dicha invocación, debe existir un mecanismo o tecnología de invocación y un contrato que defina el proceso, tal y como representamos en la siguiente figura.

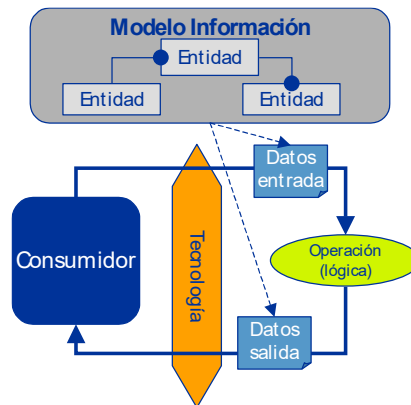


Ilustración 10. Elementos que intervienen en un servicio

En general, además, se aportan unos datos de entrada y la lógica devuelve algún tipo de resultado al consumidor y, normalmente, unos datos de salida. Estos datos de entrada y salida se adhieren a un **modelo de información**, ya sea implícito o explícito, que define una sintaxis y semántica comprensible tanto por el consumidor como por la lógica invocada.

Pues bien, la unidad mínima de lógica invocable y que hace algo útil es lo que denominaremos **operación**. En general, en una cierta área de trabajo, existen una serie de operaciones que forman un conjunto coherente y autocontenido. Esas agrupaciones coherentes es lo que denominaremos **servicio**.



Ilustración 11. Representación de un servicio

Así, con un carácter más informático y de forma algo más formal, un **servicio** es:

“Conjunto coherente de funcionalidad, constituido por una o más operaciones, autocontenido e independiente, que acepta llamadas y devuelve respuestas mediante un contrato definido de forma inequívoca”.

A partir de esta definición, podemos considerar que un servicio, en el marco de una arquitectura SOA, es:

“Una funcionalidad empaquetada como un componente reutilizable para su utilización en un proceso de negocio”

Su propósito es proporcionar información o facilitar el cambio de una información de negocio de un estado coherente a otro. Lo importante para el modelo SOA no es la tecnología usada para implementar el servicio, sino que responda a las peticiones que se le realizan y ofrezca la calidad del servicio necesaria según el contrato que ofrece.

Un servicio puede parecer a priori un componente de software, en el sentido de que desde la perspectiva del solicitante del servicio es una función autocontenida. Sin embargo, la implementación del servicio puede en realidad conllevar diferentes pasos ejecutados en diferentes equipos, y ser en sí mismo un proceso de negocio.

Por ello, un servicio puede o puede no ser un componente en el sentido software de encapsulación de funcionalidad, motivo por el que cobra interés el concepto de “orientación a servicios”, frente a la “orientación a componentes” o la “orientación a objetos”.

2.3.2 Concepto de orientación a servicios

La **orientación a servicios** ha emergido como **paradigma de diseño** que fomenta la **creación de lógica de negocio y de lógica de operación en forma de servicios**. Este paradigma consta de una serie de principios comunes de diseño que conforman unas características para los servicios que cubren perfectamente los objetivos asociados a las arquitecturas SOA y a la informática orientada a servicios.

Como otros paradigmas de diseño (diseño estructurado, diseño orientado a objetos), la orientación a servicios ofrece un modo de conseguir la llamada “**separación de conceptos**” (*separation of concerns*), mediante la que problemas de gran tamaño se descomponen en una serie de problemas individualmente identificables que son llamados “conceptos”.

El paradigma informático más extendido y reconocido es el de orientación a objetos, en el que se utilizan un conjunto de “objetos” para diseñar aplicaciones software, y en el que se utilizan técnicas de diseño como la herencia, la modularidad, el polimorfismo o la encapsulación. Su importancia en el contexto de la ingeniería software ha sido capital, y de hecho puede decirse que **la orientación a servicios le debe su existencia a la orientación a objetos**.

Las arquitecturas SOA están basadas en un modelo en el que **la lógica de la solución se encuentra distribuida**, y, por tanto, como ocurre con la orientación a objetos, conceptos como la **encapsulación**, **abstracción**, y **reutilización** son fundamentales para el diseño de unidades de lógica distribuida.

Las principales diferencias en ambos enfoques se concentran en **cómo se relacionan** unas entidades (objetos o servicios) con otras, y en el **ámbito en el que se aplican los respectivos paradigmas**.

Es importante considerar que ambos paradigmas proporcionan formas de descomposición de la lógica de una solución a diferentes niveles de abstracción, o si se prefiere, según diferentes criterios.

De esta forma, podemos decir que **el paradigma de la orientación a servicios es una evolución de anteriores aproximaciones**, aumentado y extendido para soportar las características y objetivos perseguidos con las arquitecturas SOA.

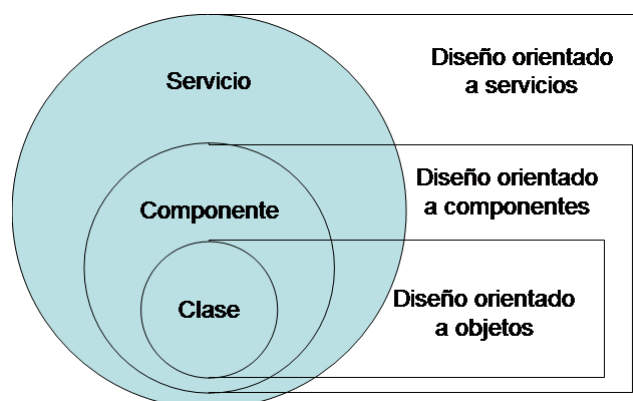


Ilustración 12. Paradigmas de diseño

El modesto ámbito inicial de la orientación a servicios, con un conjunto de principios en torno a un modelo de arquitectura enfocado en distinguir servicios como recursos reutilizables, ha cambiado por el efecto del impulso tecnológico y la tendencia a la automatización de los procesos de negocio, y

extendiéndose a nivel de toda la organización como solución de interoperabilidad y como herramienta para la independencia de proveedores.

2.3.3 Análisis, diseño y desarrollo orientado a servicios

La metodología de modelado para aplicaciones SOA se conoce como **análisis y diseño orientado a servicios**, y establece tanto un marco de trabajo para el desarrollo de software como un marco de trabajo de implantación.

Para que un proyecto SOA tenga éxito, los desarrolladores de software deben orientarse ellos mismos a la mentalidad de **crear servicios comunes** que son **orquestrados a través de un middleware** para **implementar los procesos de negocio**.

El desarrollo de sistemas usando arquitecturas SOA requiere un compromiso con este modelo en términos de planificación, herramientas e infraestructura. Por otro lado, la implementación ideal de un servicio exige resolver algunos inconvenientes técnicos inherentes a su modelo:

- Los tiempos de invocación no son despreciables, debido a los retardos de la comunicación en red, tamaño de los mensajes, etc. Esto necesariamente implica la utilización de mensajería confiable.
- La respuesta del servicio está afectada directamente por aspectos externos como problemas en la red, configuración, etc. Estos deben ser tenidos en cuenta en el diseño, desarrollándose los mecanismos de contingencia que eviten la parálisis de las aplicaciones y servicios que dependen de él.
- Comunicaciones no confiables, mensajes impredecibles, reintentos, mensajes fuera de secuencia, etc.

A su vez, cuando se usan múltiples servicios para implementar un sistema, la comunicación entre estos dificulta el control. Por ejemplo, se puede tener un servicio que llama a otros seis servicios, algunos de los cuales llaman a otros servicios, y de esta manera, muy fácilmente el sistema se vuelve inmanejable.

De esta forma, un sistema grande puede terminar con múltiples dependencias y detectar un problema de funcionalidad o rendimiento se puede volver muy complicado. Según lo expuesto, si no se cuenta con una estrategia adecuada, se puede llegar a una implementación donde exista una explosión de dependencias entre los diferentes servicios.

Será, por tanto, una decisión de diseño la elección de cubrir la lógica de un proceso de negocio mediante varios servicios de grano fino interrelacionados entre sí o mediante un servicio que centralice la definición de todo el proceso.

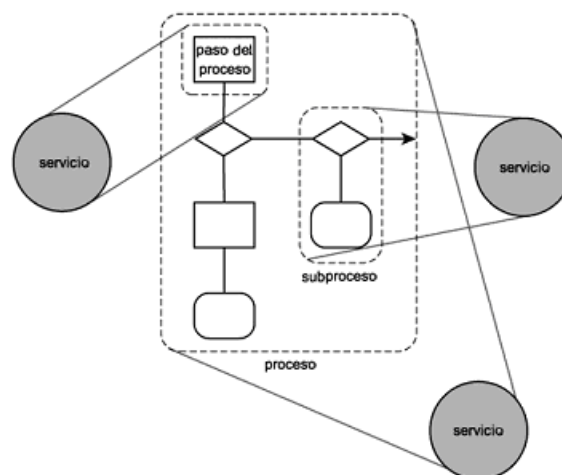


Ilustración 13. Descomposición en servicios de la lógica de un proceso de negocio

La relación entre lógica de procesos de negocio y servicios ha potenciado la sinergia entre la Gestión de Procesos de Negocio (BPM) y las arquitecturas SOA, que veremos en más detalle un poco más adelante.

Centrándonos en el proceso de diseño y desarrollo orientado a servicios destacan los siguientes pasos fundamentales:

- **Identificación de Servicios**

El proceso de identificación de servicios consiste en la combinación de tres técnicas de descomposición de dominios de aplicación:

1. **Enfoque *top-down***

En el enfoque *top-down*, un diagrama de los casos de uso de negocio proporciona la especificación para definir los servicios de negocio. De esta forma, se abstraen las áreas funcionales y subsistemas presentes en el dominio de negocio, pudiendo bajar de nivel hasta alcanzar la descomposición en flujos, procesos, subprocesos, etc.

Los casos de uso a menudo son magníficos candidatos para los servicios de negocio que se expondrán en la plataforma SOA para la organización.

2. **Enfoque *bottom-up***

En el enfoque *bottom-up*, los sistemas y aplicaciones preexistentes de la organización son analizados, y como resultado del análisis, aquellos que sean considerados candidatos viables para proporcionar la implementación de la funcionalidad de los servicios que compondrán el proceso de negocio, son seleccionados.

En este proceso, las aplicaciones empaquetadas y heredadas tienden a verse como un todo, aunque en ocasiones es necesario remodularizarlas para ofrecer la funcionalidad de servicio necesaria.

3. **Enfoque *middle-out***

Finalmente, el enfoque *middle-out* consiste en modelar y definir aquellos servicios no identificados mediante los dos enfoques anteriores

- **Clasificación o categorización de servicios**

Una vez identificados los servicios, comienza esta actividad de clasificación de los servicios en una jerarquía y estructura que refleje su naturaleza compuesta y fractal.

Cada servicio tiene diferentes usos y propósitos, de forma que podemos distinguir entre servicios básicos de negocio, servicios de infraestructura software, servicios sectoriales de negocio, etc.

A partir de ellos, considerados como servicios atómicos, es posible componer y orquestar otros servicios de más alto nivel, de forma que algunos servicios se componen de servicios y componentes de grano más fino, escenario en el que la clasificación ayuda a determinar las interdependencias al tiempo que evita la proliferación innecesaria de servicios con funcionalidad no reutilizable.

- **Análisis de subsistemas**

Como se ha comentado, dentro de la primera fase de identificación de servicios, se alcanza una descomposición en subsistemas al aplicar el enfoque *top-down*.

La presente actividad toma los subsistemas identificados en la primera fase y especifica las interdependencias y los flujos entre los mismos. Asimismo, los casos de uso identificados como servicios son expuestos en la interfaz de los subsistemas identificados.

De esta forma, el análisis de subsistemas consiste en crear modelos de objetos para representar el funcionamiento interno de los subsistemas y el diseño de los servicios que se exponen que llevarán a efecto dicho funcionamiento.

- **Especificación de componentes**

En esta actividad se especifican los componentes que implementan los servicios y sus detalles:

- Información
- Reglas de negocio
- Servicios que implementa
- Perfil configurable

Las especificaciones sobre los eventos y el intercambio de mensajes y la gestión de los componentes también se realizan en esta fase.

- **Asignación de servicios**

La asignación de servicios consiste en vincular los servicios a los subsistemas previamente identificados, vinculando los servicios y componentes que los implementan.

- **Implementación del servicio**

Finalmente, esta fase reconoce que el software que implementa un servicio dado deberá ser seleccionado o bien construido a medida. Las opciones de implementación de un servicio incluyen la integración, transformación o suscripción de funcionalidades implementadas bajo diferentes tecnologías, paradigmas y/o sistemas (servicios web, objetos Java, objetos CORBA, sistemas heredados – *legacy systems*, etc.).

Así, en este paso, se toma la decisión de qué sistema heredado será utilizado para dar un servicio dado, qué servicios serán implementados “desde cero”, qué aspectos de seguridad deberán tenerse en cuenta, etc.

Se aprovechan de nuevo aquí los esfuerzos realizados en la primera fase de identificación de servicios, sacando provecho de las descomposiciones *top-down* y *bottom-up* realizadas, de forma que se alinean los servicios con su funcionalidad de negocio.

De forma resumida los reflejamos en la siguiente figura.

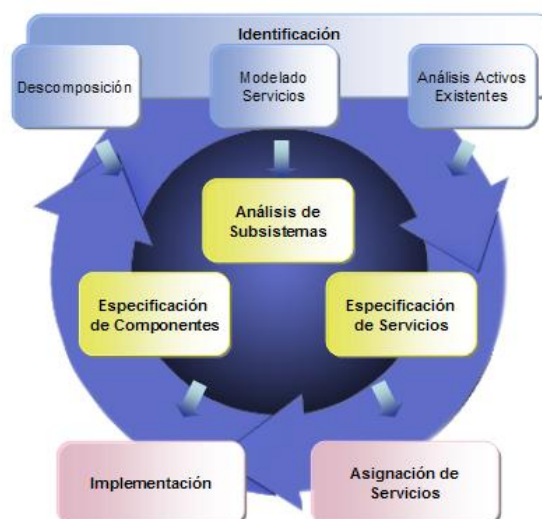


Ilustración 14. El proceso de identificación de los servicios

2.4 SOA y Web Services

El paradigma SOA es muy general y de carácter muy abstracto, sin que establezca una tecnología específica de implementación del mismo.

La elección tecnológica conlleva la selección de estándares, protocolos, lenguajes de programación, infraestructura específica, etc., que permiten llevar el paradigma a la práctica.

Una de las tecnologías más destacadas y que con más éxito está llevando a la práctica el paradigma conceptual de SOA son los Web Services (que llamaremos también indistintamente a lo largo del documento como Servicios Web).

Por este motivo, a continuación, se exponen los aspectos generales de la tecnología de Servicios Web que la configuran como la mejor alternativa para la implementación de una arquitectura SOA. No obstante, se hará mucho hincapié en que Servicios Web y SOA son diferentes conceptos y que no debe confundirse donde empieza uno y acaba el otro.

2.4.1 Arquitectura de servicios web

Veámos en el apartado 2.3.1 una definición muy general del concepto de servicio como una funcionalidad ofrecida entre un proveedor y un requirente. Para el caso de un **Servicio Web** (*Web Service*), el organismo W3C, nos facilita una definición más específica:

“Un servicio web es un sistema software diseñado para soportar la interacción máquina-a-máquina a través de una red y de forma interoperable”

Un servicio web cuenta con una interfaz descrita en un formato procesable por un equipo informático (específicamente en WSDL), a través de la cual es posible interactuar con el mismo mediante el intercambio de mensajes SOAP, típicamente transmitidos usando serialización XML sobre HTTP juntamente con otros estándares web.

Para conocer el formato, el solicitante consulta a un intermediador, un registro de servicios, donde los proveedores de servicios web publican sus formatos y a los solicitantes se les ofrece un servicio de consulta similar a unas “páginas amarillas”.

Podemos ver el esquema de esta arquitectura y el intercambio de mensajes de forma resumida en la siguiente figura.

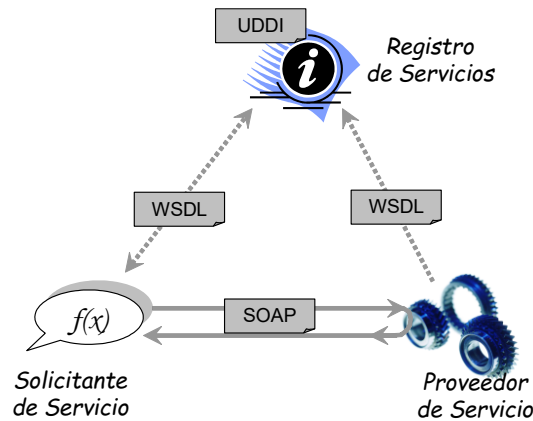


Ilustración 15. Esquema de la arquitectura de servicios web

Sobre la básica arquitectura representada en la figura anterior, el organismo W3C ha trabajado en la definición de una arquitectura conceptual de servicios web (<http://www.w3.org/TR/ws-arch/>) que proporciona un modelo conceptual y un contexto para entender qué se esconde detrás del concepto de *Web Service*.

La arquitectura de servicios web propuesta por el W3C no pretende especificar cómo se implementan los servicios web, ni impone restricciones sobre cómo deben relacionarse, combinarse o coordinarse entre sí, sino que define modelos conceptuales que ilustran las principales dimensiones a considerar (mensajería, recursos, servicios y políticas) y conforma los principales aspectos a considerar a la hora de utilizar Web Services (interoperabilidad, fiabilidad, integración, seguridad, transaccionalidad, extensibilidad, gestión y provisión).

2.4.2 Web Services como alternativa para implementar arquitecturas SOA

Un error muy común relacionado con las arquitecturas SOA es la vinculación directa que se realiza con la tecnología de Web Services: **los Web Services/Servicios Web NO son SOA**.

Como ejemplo de lo anterior, baste decir que una arquitectura SOA puede llegar a ser implementada, sin que para ello exista un solo Servicio Web, o de forma inversa, se pueden implementar Servicios Web sin que exista una plataforma o infraestructura SOA que los soporte por detrás.

La diferencia entre ambos términos es conceptual: mientras que **SOA define el qué**, los **Servicios Web definen el cómo**. O, dicho de otro modo, al igual que es posible aplicar el paradigma de orientación a objetos con Java o C++, el paradigma de orientación a servicios no es exclusivo de los servicios web.

El paradigma de diseño establecido por la “orientación a servicios” pretende ser agnóstico respecto a la implementación, y tiene como meta el establecer un conjunto interrelacionado de principios para lograr los beneficios ya comentados con independencia de cómo se manifieste su implementación el mundo real.

Por otra parte, el concepto de “servicio” en SOA se basa en que los servicios desarrollados son “Servicios de Negocio”, sean desarrollados mediante una tecnología como “servicios web” o bien en otras tecnologías disponibles para ello. En una arquitectura SOA, el foco de interés se pone en el modelado de procesos a partir de piezas reutilizables que configuran servicios de negocio, abstrayéndose de la tecnología y forma en la que esos servicios son implementados.

Uno de los principios de SOA vistos anteriormente es la reusabilidad, ¿cómo se logra si no se utilizan servicios web? Se logra no por la posibilidad de reutilizar los componentes software que implementan dichos servicios sino en la medida en que los Servicios de Negocio son involucrados en nuevos y más procesos.

Lo anterior indica que todo nuevo servicio de negocio desarrollado debe ser puesto a disposición y en conocimiento de las áreas de diseño de procesos. Puesto que el servicio de negocio está definido en

términos del negocio y no de tecnología, es fácil hacer referencia e incorporarlo a sus diseños para las áreas de negocio.

La abstracción entre el servicio de negocio y los diferentes servicios de aplicación y tecnología (que, de nuevo, pueden estar o no desarrollados como servicios web), así como los términos de negocio que se enmarcan, permiten que cada vez que se haga uso de un servicio de negocio, la tarea de desarrollo disminuya, ya que la lógica con la que fue implementado esta lista, para ser utilizada por quien lo requiera.

Aclarada la diferencia entre los servicios web y las arquitecturas SOA, no obstante, hay que dejar claro que los servicios web, por sus especiales características de interoperabilidad, reutilización, modularidad, etc., se ajustan perfectamente a las necesidades y principios de las arquitecturas SOA. Todo ello, añadido a la **universalización de los Web Services**, implementados por la mayoría de empresas de software para un enorme espectro de plataformas y sistemas operativos, lo han convertido en la **aproximación más extendida de implementar arquitecturas SOA**.

Veremos, a continuación, los estándares y protocolos de servicios web más extendidos y utilizados en la implementación de arquitecturas SOA.

2.4.3 Estándares y protocolos de Web Services en las arquitecturas SOA

Las arquitecturas de servicios web involucran a muchas tecnologías relacionadas, y hay numerosas formas de construir y utilizar servicios web.

Según la definición de Arquitectura de Web Services del W3C (<http://www.w3.org/TR/ws-arch/>), la lista de estándares y protocolos básicos es la que se muestra en la siguiente figura.

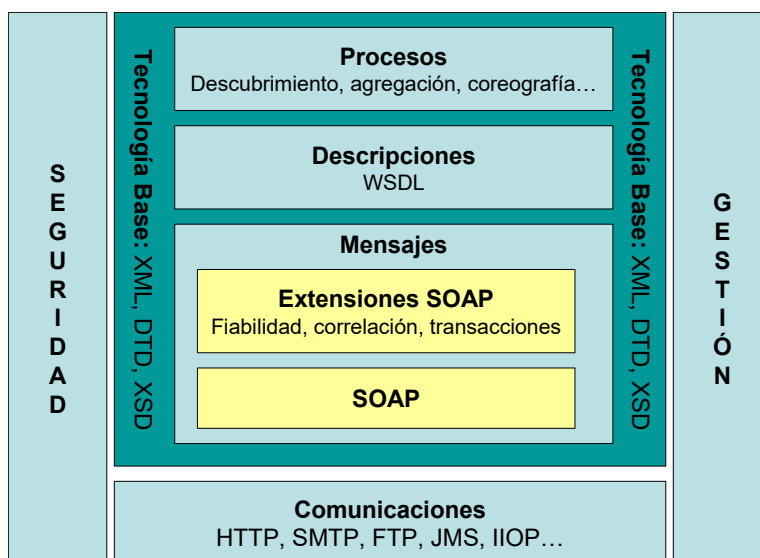


Ilustración 16. Estándares de la arquitectura de servicios web del W3C

En otros temas del presente temario se describen en mayor detalle algunos de los principales estándares y protocolos en relación con los Web Services representados en la figura, y que se incluyen de forma resumida a continuación por completitud:

- **XML** (*eXtensible Markup Language*), estándar de W3C (<http://www.w3.org/xml/>) actualmente en su versión 1.1, es un lenguaje de marcado que permite describir información o contenido de forma completamente separada de su presentación o formato. Junto a él, hay un amplio conjunto de estándares para la definición de lenguajes y validación de documentos (DTD, XSD) y para la transformación y presentación de los mismos (XSL-FO, XSLT).
- **SOAP** (*Simple Object Access Protocol*), estándar del W3C (<http://www.w3.org/TR/soap/>) actualmente en su versión 1.2, que define una gramática XML que describe el formato de los

documentos XML a intercambiar entre el solicitante y el proveedor del servicio, en concreto, el formato en el que se “ensobran” las peticiones y respuestas entre solicitante y proveedor.

- **WSDL** (*Web Services Description Language*), actualmente en su versión 2.0 y también estándar del W3C (<https://www.w3.org/TR/wsdl20/>), que define una gramática XML que permite describir la interfaz de un servicio web, es decir, las funciones y parámetros de entrada y salida necesarios para la invocación de los servicios.
- **UDDI** (*Universal Description Discovery and Integration*), en su versión 3.0.2 y perteneciente a OASIS (<http://uddi.xml.org/>), que configura un servicio de registro o servicio de directorio donde los proveedores de servicios publican sus servicios web y los solicitantes de servicios buscan y encuentran los servicios web acordes a sus necesidades, de forma parecida a como funciona un servicio de “páginas amarillas”.

Junto a los anteriores estándares básicos, es interesante considerar el papel de los perfiles de *WS-Interoperability* (WS-I), cuyo propósito es promover la interoperabilidad de los servicios web entre plataformas, sistemas operativos y lenguajes de programación.

La organización WS-I centra su labor en la integración de especificaciones, proporciona aplicaciones de ejemplo, guías y mejores prácticas recomendadas, así como recursos de apoyo y herramientas, trabajo cuyos resultados organiza en torno al concepto de “perfiles”. Actualmente se encuentran a disposición los siguientes perfiles:

- **WS-I Basic Profile**, convenciones sobre mensajería, descripción, descubrimiento y seguridad en los servicios web, introduciendo restricciones y aclaraciones y guías de utilización conjunta de los estándares SOAP, WSDL, UDDI, XML, XML Schema y HTTP.
- **WS-I Basic Security Profile (BSP)**, extensión del Basic Profile que cubre la interoperabilidad en términos de seguridad, en la capa de transporte (HTTP sobre TLS), seguridad en la mensajería SOAP y estándar WS-Security.
- **WS-I Reliable Secure Profile**, relacionado con la interoperabilidad de las especificaciones WS-Reliable Messaging, WS-Reliable Exchange y WS-Secure Conversation.

Finalmente, en el contexto de las arquitecturas de servicios web y en su relación con los niveles y la definición de una arquitectura de referencia SOA, es interesante dar cuenta de los innumerables estándares vinculados a los servicios web, siempre en continuo crecimiento:

Contexto	Estándares
Transporte	BEEP (<i>Blocks Extensible Exchange Protocol</i>)
Mensajería (junto a SOAP)	WS-Addressing WS-Notification (WS-BrokeredNotification, WS-BaseNotification, WS-Topics) WS-Attachments Profile 1.0 MTOM Serialization Policy Assertion (WS-MTOMPolicy) Version 1.0
Descripción y descubrimiento (junto a UDDI y WSDL)	WS-Semantics (WSDL-S) WS-Metadata Exchange WS-Policy Assertions Language WS-Policy Attachment WS-Policy Framework WS-Resource Framework
Fiabilidad	WS-Reliable Messaging WS-Reliable Exchange WS-RM Policy Assertion
Transaccionalidad	WS-Transaction WS-Atomic Transaction WS-Business Activity WS-Coordination
Seguridad	WS-Federation Language (Active and Passive Requester Profile) WS-Secure Conversation Language WS-Secure Exchange WS-Security 1.0 WS-Security Addendum WS-Security Kerberos Binding WS-Security Policy WS-Trust Security Assertion Markup Language (SAML)
Procesos de negocio	Business Process Execution Language for Web Services (WS-BPEL) V2.0 WS-BPEL Extension for People (BPEL4People)

Contexto	Estándares
Coordinación	WS-CAF WS-CDL
Gestión	WS-Distributed Management WS-Manageability WS-ResourceTransfer WS-Service Registry and Repository

Considerando los estándares y protocolos anteriores, una arquitectura SOA conformada mediante Servicios Web contará con el siguiente aspecto:

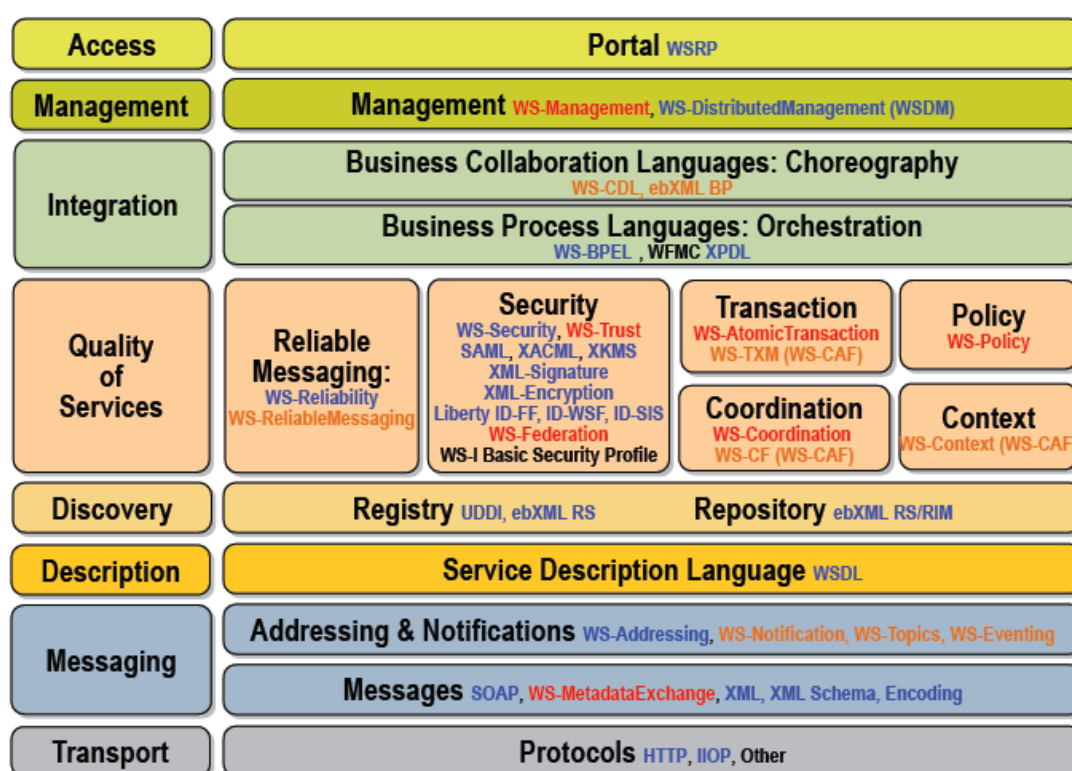


Ilustración 17. Estándares y protocolos de una arquitectura SOA conformada mediante servicios web

La elección entre las diferentes alternativas tecnológicas a los mismos, así como la posible utilización de otros estándares, será objeto de la definición de la arquitectura de referencia de nuestra implementación de arquitectura SOA, aspecto que retomaremos más adelante en el apartado 2.6.

2.5 SOA y Procesos de Negocio

2.5.1 Procesos de Negocio y BPM

El estándar **Business Process Management (BPM)** es utilizado por un gran número de empresas y organizaciones como una forma efectiva de mejorar su eficacia, rendimiento y su eficiencia. Detrás de BPM, como su nombre indica, se esconde una disciplina empresarial cuyo objetivo es **mejorar la eficiencia** a través de la **administración y gestión sistemática de los procesos de negocio**, que se deben modelar, automatizar, integrar, monitorizar y optimizar de forma continua.

Un **proceso de negocio** no es más que un **conjunto de tareas o actividades** relacionadas lógicamente que **permiten crear valor** transformando una entrada en una salida logrando así un resultado de negocio definido. Las entradas son prerequisites que deben tenerse antes de que una función pueda ser aplicada.

Cuando una función es aplicada a las entradas de un método, tendremos ciertas salidas resultantes, siendo ambas entradas y salidas entidades o información que pueden ser transformadas por personas, máquinas y Sistemas de Información o por ambos.

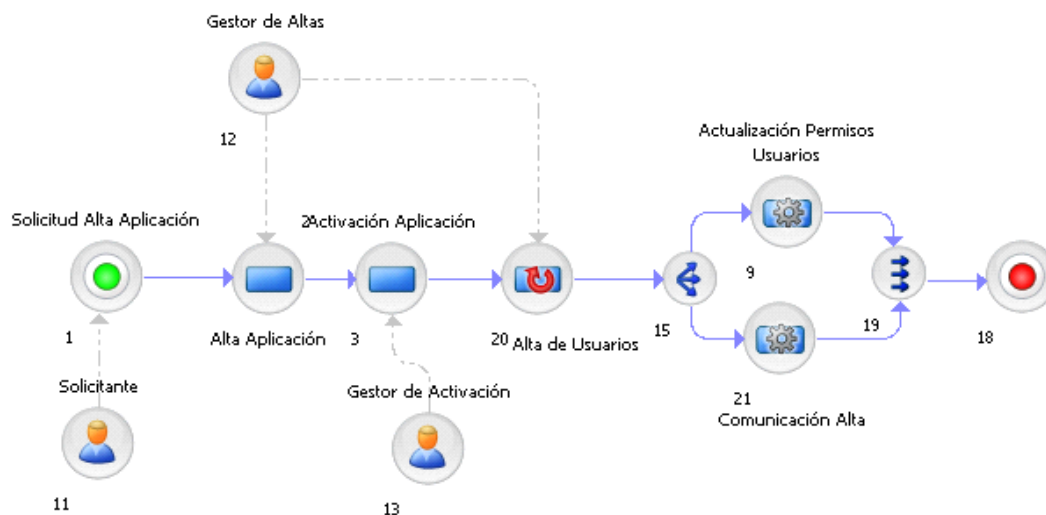


Ilustración 18. Diagrama de un proceso de negocio

La **Gestión de Procesos de Negocio** en una organización es, por tanto, el área o campo de conocimiento intersección de la Gestión y las Tecnologías de la Información, que abarca las técnicas, métodos y herramientas para diseñar, representar, controlar y analizar los procesos de negocio operacionales que involucran a personas, organizaciones, aplicaciones y documentos en el contexto de una organización.

Se habla de “*procesos de negocio operacionales*” para enfatizar con ello que se hace referencia a procesos que se realizan de forma repetitiva en el día a día de las organizaciones, por contraposición a procesos estratégicos de decisión que se realizan en los niveles directivos de las organizaciones.

Por su parte, el concepto de BPM no tiene relación con el concepto de *reingeniería de procesos* (BPR), orientado a realizar cambios drásticos en los procesos de negocio de una organización para mejorar la eficiencia. El concepto de eficiencia se presenta en ambos, pero en el BPM el objetivo es apoyar la continua evolución de los procesos incorporando para ello métodos de gestión junto a Tecnología de la Información como soporte.

En la práctica, una organización comienza un proyecto de BPM con el objetivo de optimizar un área que se ha identificado como área de mejora.

A partir del comienzo del mismo, se ponen en marcha las diferentes actividades involucradas en la Gestión de Procesos:

- **Diseño:** abarca el diseño y la captura de los procesos de negocio existentes en la organización, sus reglas de negocio, sus plazos, decisiones, etc. Conlleva el uso de editores gráficos y la realización de simulaciones de comportamiento para medir los costes y tiempos medios.
- **Ejecución:** documentar un proceso es muy complicado, y la automatización del mismo tradicionalmente conllevaba construir una aplicación ex profeso que ejecutara los diferentes pasos. Ahora, el proceso se traduce directamente en un lenguaje-máquina, un lenguaje de definición del proceso desde el modelo de diseño que se ejecuta sobre un motor de ejecución

de cualquier equipo, permitiendo la participación humana sólo en aquellos pasos donde sea oportuna su actuación.

La ejecución de las tareas puede conllevar ejecución de lógica en diferentes equipos y según diferentes paradigmas (de forma síncrona, asíncrona, etc.), por lo que es necesaria para la implementación de las mismas un mecanismo flexible e interoperable de petición/respuesta.

- **Monitorización:** el rendimiento de la ejecución, los cuellos de botella de los procesos, etc., son elementos a vigilar durante la ejecución de los procesos, de forma que se establecen indicadores a analizar y evaluar, tanto de forma reactiva como proactiva.



Ilustración 19. Actividades involucradas en la gestión de procesos

Las opciones tecnológicas para cubrir las fases anteriores son variadas, si bien la tendencia actual, como en otras disciplinas, se ha orientado al uso del lenguaje XML, y, por sus particulares sinergias, en la fase de ejecución, al uso de las arquitecturas SOA como infraestructura sobre la que disponer de servicios como piezas con las que construir los procesos de negocio.

2.5.2 Organismos de estandarización y estándares

Los organismos de estandarización más relevantes en este campo son:

- **BPMI:** En agosto de 2000 se crea la *Business Process Management Initiative* (BPMI.org), una organización sin fines de lucro que también agrupa a varias instituciones y tiene como misión promover el uso de BPM.

BPMI.org desarrolla especificaciones abiertas para el diseño, ejecución, mantenimiento y optimización de procesos de negocio, y en particular es el autor de una especificación para la notación (BPMN) y un lenguaje de modelado de procesos de negocios (BPML). BPML, nacido en 2001, ha sido una gran fuente de inspiración para el estándar BPEL de OASIS.

- **Workflow Management Coalition (WfMC):** Los principales esfuerzos en el desarrollo y promoción de estándares en el campo de los sistemas de workflow (*Workflow Management Systems* - WMS) vinieron de parte de la WfMC, organización internacional, sin fines de lucro, que se establece en 1993 y reúne a un grupo de vendedores, usuarios, analistas y grupos de investigación.

Su principal objetivo es promover el uso de WMS mediante el establecimiento de estándares que faciliten la creación, desarrollo y análisis de estos sistemas, para lo que ha definido un Modelo de Referencia que describe la arquitectura básica de un WMS y que constituyen el material básico de referencia. Junto a la labor anterior, la WfMC es la promotora del estándar XPD, nacido en 2002 a partir de un lenguaje de ejecución precursor.

- **OASIS:** es un consorcio internacional sin fines de lucro que orienta el desarrollo, la convergencia y la adopción de estándares de comercio electrónico y de servicios web. En el contexto del BPM, se hizo cargo a partir de 2003 de la evolución y desarrollo del lenguaje BPEL (Business Process Execution Language), inicialmente creado por Microsoft e IBM en respuesta a la iniciativa de BPML, y que se ha convertido en el estándar de facto de la industria.
- **Object Management Group (OMG):** un caso curioso a mencionar es el de OMG, que se unió a los esfuerzos de la WfMC en la propuesta de estándares en 1997.

OMG es también una organización internacional sin fines de lucro fundada en 1989, cuyo principal objetivo se centra en promover la teoría y práctica de la tecnología orientada a objetos en el desarrollo de software. Aunque fuera de su ámbito de aplicación más inmediato, en 1997 se inicia un acercamiento hacia los WMSs, y este acercamiento de OMG conlleva a la publicación de una primera especificación, basada en los estándares de la WfMC, que establece requisitos de interoperabilidad entre distintos WMS en un entorno CORBA.

En junio de 2005, se produjo una fusión estratégica entre BPMI.org y OMG, acontecimiento de relevancia si se tiene en cuenta que dos de las tecnologías de notación más difundidas hoy por hoy son BPMN (de la BPMI.org) y los diagramas de Actividad de UML.

Y los estándares más relevantes son los siguientes:

Nombre	Organismo	Campo de aplicación	Notación	Formato intercambio	
XPDL	WfMC	Lenguaje de ejecución	No	Sí	http://www.xpdl.org/
BPEL	OASIS	Lenguaje de ejecución	No	Sí	http://www.oasis-open.org/committees/wsbpel/
BPMN	BPMI	Análisis de Procesos de Negocio	Sí	No	http://www.bpmn.org/
BPML	BPMI	Lenguaje de ejecución	No	Sí	[Discontinuado]

2.5.3 BPM y SOA

Desde un punto de vista de negocio, la **combinación de BPM y SOA** beneficia de forma conjunta a los profesionales de Tecnologías de la Información y a los usuarios Gestores de Negocio:

1. Los profesionales TIC cuentan con un nuevo paradigma que permite la **reusabilidad** y el **bajo acoplamiento, mejorando la calidad y mantenibilidad** del software y las aplicaciones desarrolladas.
2. Los Gestores de Negocio utilizan una tecnología que les permite **agilizar y gestionar de forma más eficiente los procesos de negocio** de la organización.

Partiendo de la idea clara de que una arquitectura SOA es perfectamente útil sin necesidad de contar con una infraestructura BPM como soporte, y que igualmente se puede disponer de BPM sin contar con SOA, ambas tecnologías demuestran tener una perfecta complementariedad.

A través de BPM, se modelan procesos de negocio como un conjunto de tareas de procesamiento individuales, típicamente implementadas como servicios en el contexto de una organización.

De esta forma, BPM ayuda en la creación de los modelos de proceso, pero también en la automatización de los flujos resultantes y en la coordinación con la intervención humana y la forma de invocación de los servicios comentados.

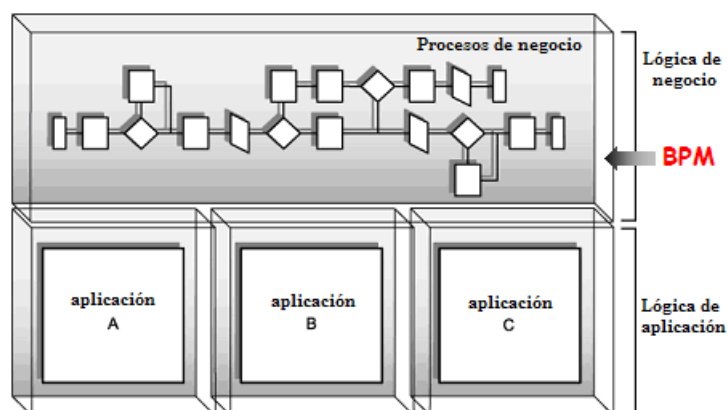


Ilustración 20. Ejemplo de agrupación de los procesos de negocio en aplicaciones

Por otro lado, las arquitecturas SOA aportan al modelado de procesos un nuevo concepto denominado **composición de servicios**, que no sólo permite **modelar el proceso de negocio**, sino que también maximiza las potencialidades que las arquitecturas SOA brindan a través de la **integración de datos y aplicaciones en forma de servicios**.

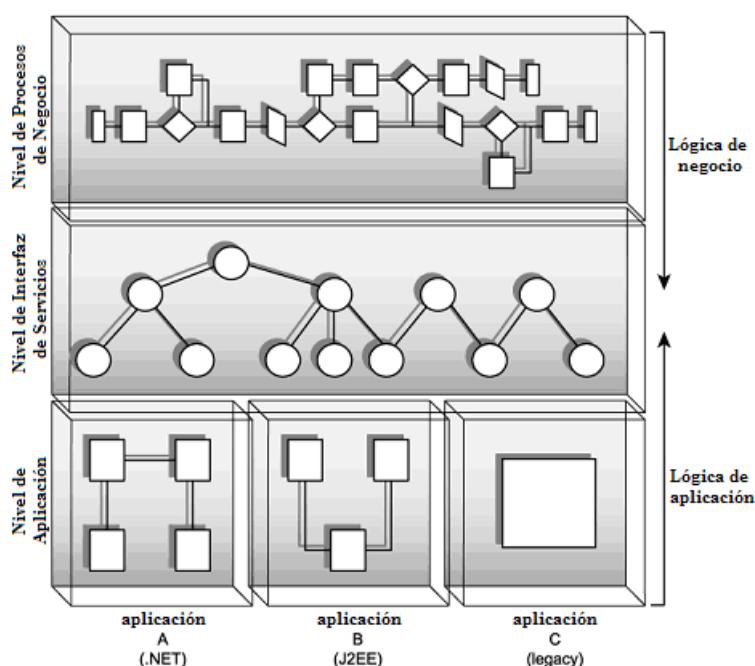


Ilustración 21. Introducción de una capa de servicios

La composición de servicios implica establecer un **mecanismo que permita a dos o más servicios cooperar entre sí** para resolver requisitos que van más allá del alcance de sus capacidades individuales. Algunos de los requisitos básicos que se deben cumplir son:

- **Interacciones asíncronas con servicios:** se da así la posibilidad de crear procesos que transcurran durante largos períodos de tiempo.
- **Interacciones simultáneas entre servicios:** esto introduce el procesamiento en paralelo, lo cual redundará en un aumento considerable de la eficiencia.
- **Manejo de excepciones:** un proceso basado en múltiples servicios puede fallar en su ejecución si al menos uno de sus componentes falla. Se debe tener en cuenta la ocurrencia

de errores y proveer mecanismos para manejarlos.

- **Integridad transaccional:** un proceso de larga duración puede eventualmente fallar, en este caso puede requerirse que parte de las acciones ya efectuadas se deshagan.

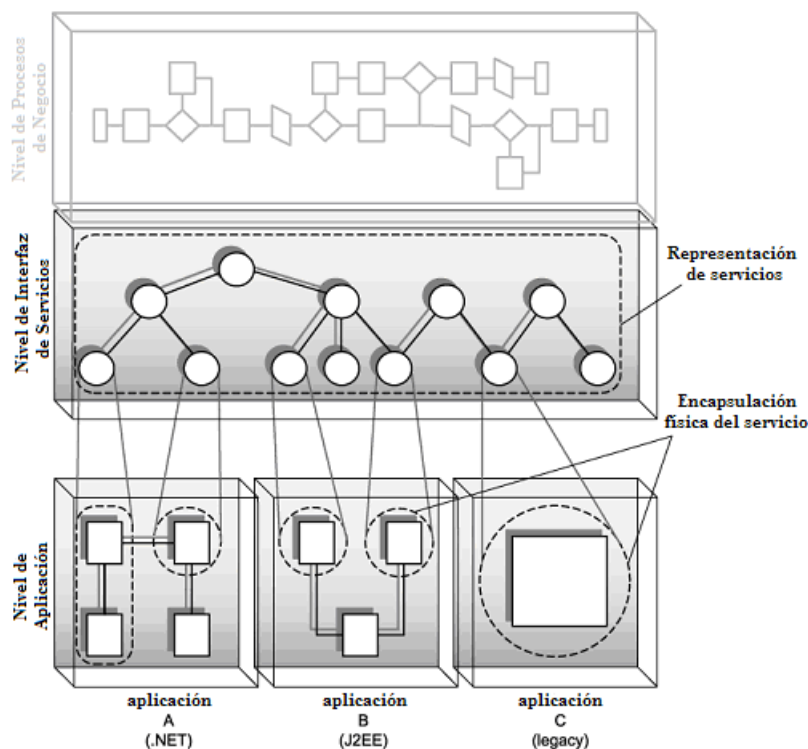


Ilustración 22. Ejemplo de una posible implementación de servicios

Dos términos comúnmente usados para referirse a la colaboración entre servicios son **orquestración** y **coreografía**. Ambos están directamente relacionados con la composición, pero se enfocan en aspectos complementarios de la interacción entre servicios:

- **Orquestración:** Un proceso se puede considerar una orquestración de servicios cuando es controlado totalmente por una única entidad.

Este proceso define completamente las interacciones con los servicios componentes y la lógica requerida para conducir correctamente esas interacciones, y sólo la entidad que está orquestrando el proceso conoce el flujo de control e información que sigue el proceso que se está orquestrando.

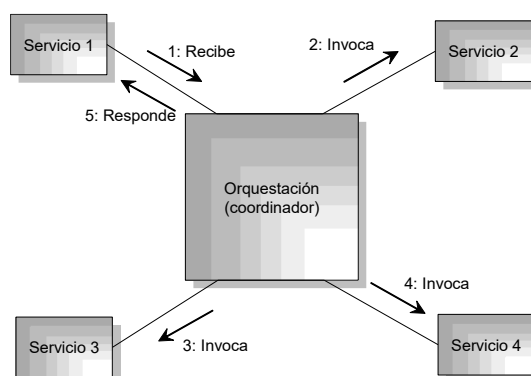


Ilustración 23. Ejemplo de orquestración de servicios

De esta manera, se crea un proceso que utiliza diferentes servicios manipulando la información que fluye entre ellos, convirtiendo, por ejemplo, los datos de salida de algunos

servicios en datos de entrada de otro. Aquí, cada entidad que participa implementa y controla su propio proceso.

- **Coreografía:** Un proceso es una coreografía de servicios cuando define las colaboraciones entre cualquier tipo de aplicaciones componentes, independientemente del lenguaje o plataforma en el que estén definidas las mismas.

Un proceso de coreografía no es controlado por uno solo de los participantes, y a diferencia de la orquestación, puede verse como un proceso público y no ejecutable. Es público porque define un comportamiento común que todas las entidades participantes deben conocer, y es no ejecutable porque está pensado para verse más bien como un protocolo de negocio que dicta las reglas para que dichas entidades puedan interactuar entre sí.

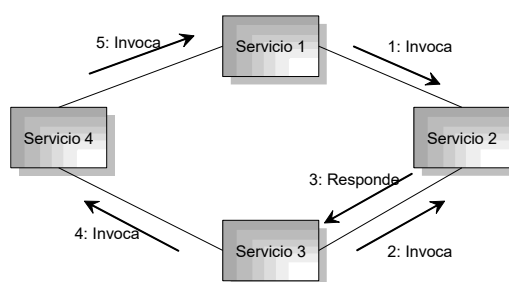


Ilustración 24. Ejemplo de coreografía de servicios

Ya sea mediante uno u otro mecanismo, las arquitecturas SOA aportan a BPM la posibilidad de implementar las tareas y actividades que conforman los procesos de negocio mediante servicios, lo que permite una gran flexibilidad al poder implementar cada servicio de forma independiente.

Además, la distribución de los servicios, connatural a las arquitecturas SOA, permite que múltiples agentes se integren más fácilmente en los procesos de negocio.

De esta forma, mientras que a “nivel SOA” se resuelven la conectividad y comunicación a nivel técnico, como los servicios se implementan, etc., a “nivel BPM” se resuelven los aspectos de negocio, la secuenciación de actividades, las reglas de negocio aplicables, etc., y ambos constituyen la pareja perfecta.

2.6 Arquitectura de Referencia SOA

2.6.1 ¿Qué es una arquitectura de referencia SOA?

En los apartados anteriores, se ha visto de forma conceptual qué es la arquitectura SOA, y se ha constatado que define un estilo de arquitectura y no una arquitectura determinada, siendo esta última la representación que podemos alcanzar a través del uso de tecnologías específicas como los Web Services/Servicios Web.

Lo anterior es el punto de partida de la necesidad de definir una **arquitectura de referencia SOA**, como el marco conceptual objetivo para la implementación física de una arquitectura SOA para una empresa, organización o línea de actividad. Podemos considerar la arquitectura de referencia como el “estado futuro” o “visión futura” que nos permite construir la hoja de ruta entre nuestro estado actual tecnológico a un estado futuro con proyectos desplegados sobre infraestructura SOA.

El papel de la arquitectura de referencia SOA es **identificar los aspectos fundamentales a considerar** y las **soluciones abstractas** que conforman los **principales problemas a la hora de afrontar un proyecto de implementación e implantación de una arquitectura SOA**.



Ilustración 25. Aspectos a considerar en una arquitectura de referencia SOA

Por otra parte, no toda solución de arquitectura es válida para todos los casos, sino que dependerá del tipo de proyectos a realizar, el volumen de negocio y/o actividad de nuestra organización, etc.

De esta forma, una arquitectura de referencia nos permite establecer un patrón general de implementación, que tome en consideración las funcionalidades necesarias en una correcta arquitectura SOA y sea el resultado de un conjunto de deducciones lógicas basadas en el estado del arte de las infraestructuras y proyectos software.

En este sentido, la arquitectura de referencia abordará, entre otras, las siguientes problemáticas:

- Paradigmas (modelos de invocación) de servicios aplicables
 - Comunicación Síncrona versus Asíncrona
 - Comunicación Bloqueante versus No Bloqueante
 - *Request/Reply* versus *Publish/Subscribe*
- Tecnologías y estándares
 - Web Services
 - Mensajería basada en MOM (JMS, MSMQ)
 - APIs nativas (bien en local por invocación directa de métodos de objetos, o en red conforme a modelo DOM)
 - RMI
 - RMI-IIOP
 - DCOM
 - Conectores
- Definición de servicios y establecimiento de contratos y SLAs
- Modelos de información de intercambio
- Registro y descubrimiento de servicios

- Anatomía de servicios y operaciones (elementos constituyentes)
- Normalización de servicios
- Problemáticas de coordinación de servicios
 - Transacciones
 - Seguridad
 - Errores
 - Trazas

2.6.2 Contenido de una arquitectura de referencia

Una vez descrito lo que entendemos por una arquitectura de referencia, es interesante que la dotemos de contenido, para lo cual haremos el ejercicio de describir las coincidencias encontradas entre las numerosas arquitecturas de referencia elaboradas por las principales empresas y consultoras del sector: IBM, Gartner, IDG, etc.

El punto de comienzo para construir una arquitectura de referencia es considerar los servicios. Una arquitectura SOA está construida en torno al concepto de **servicios de negocio**, como piezas lógicas reutilizables diseñadas para ejecutar una tarea o parte de un proceso de negocio, y que cuenta con interfaces de acceso y de invocación claramente establecidos.

La primera capacidad que debemos dotar a nuestra arquitectura de referencia SOA tiene que ver con la **gestión de servicios**, su creación, eliminación, etc.

De forma anexa, una vez que los servicios son gestionados de forma individual, la potencia de SOA reside de la ejecución encadenada y coordinada de servicios, para lo que de forma común para obtener un bajo acoplamiento se piensa en una **plataforma de mensajería** independiente de la plataforma.

No obstante, aunque los servicios puedan comunicarse a través de un bus de mensajes, la reutilización sólo es posible cuando es fácil localizar los servicios existentes y lo que hacen, de forma que necesitamos un **repositorio o registro de servicios**.

Un aspecto importante de los servicios SOA es que están claramente relacionados como piezas de una funcionalidad de negocio, más allá de ser piezas de código. Esto permite que sean ensamblados en un **proceso de negocio utilizando técnicas de orquestación o coreografía** a través de la definición de reglas y políticas de negocio que controlen el flujo.

El soporte para realizar dicha labor será una **herramienta BPM** que permita abstraer la tecnología subyacente y analizar el sistema de forma global en términos de negocio, prediciendo o simulando las prestaciones del sistema y los impactos de los cambios en los procesos. De esta forma, los cambios derivados de las necesidades de los usuarios o de negocio serán realizados más eficaz y rápidamente, sustituyendo las “piezas del puzle” que conforman los servicios subyacentes con mínima interrupción y disrupción del servicio al usuario.

Finalmente, nos encontramos con las **tecnologías de interacción y presentación al usuario**, como portales personalizados o aplicaciones para dispositivos específicos (móviles, PDAs, TV). Si bien los posibles cambios necesarios para adaptar la interfaz de usuario pueden imponer alguna limitación y restricción a las aplicaciones SOA, esa limitación se verá reducida por las actuales **herramientas de adaptación al canal** que permiten abstraer la presentación final de los desarrollos que soportan la funcionalidad de negocio.

Finalmente, de forma transversal a lo expuesto, una vez disponemos de un registro de servicios, un bus o plataforma de mensajería para la comunicación entre los mismos, una gestión básica de los servicios, una definición de procesos de negocio, y un nivel de presentación al usuario, se hace necesario disponer de un **nivel transversal** que ofrezca los **aspectos de calidad de servicio, gestión, seguridad, control de la transaccionalidad, monitorización** y cualesquiera otras funciones que permitan la **gobernanza** de la plataforma que ofrece la infraestructura SOA.

Con todo lo anterior, podemos representar nuestra arquitectura de referencia SOA como una arquitectura en diferentes niveles cómo la representada en la siguiente figura.

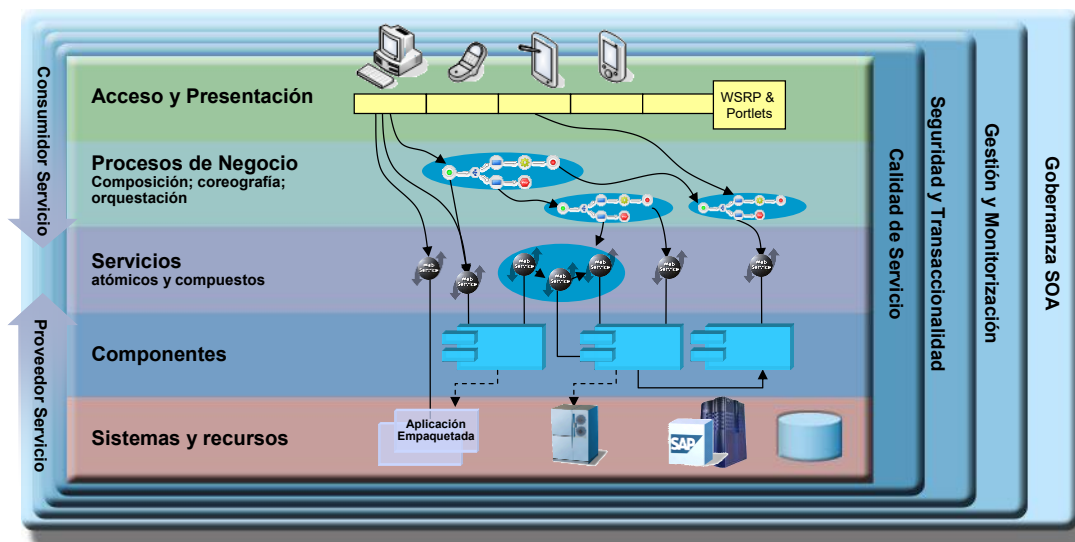


Ilustración 26. Niveles de una arquitectura de referencia SOA

Veamos una pequeña explicación de cada nivel:

Nivel 1: Capa de recursos

Consiste en las aplicaciones existentes ya construidas, ya sea mediante sistemas legados o heredados (*legacy systems*), que incluyen habitualmente sistemas CRM y ERP, como mediante otras aplicaciones de negocio y sistemas implementados con diferentes paradigmas y tecnologías.

Las arquitecturas SOA permiten potenciar estos sistemas y desarrollos existentes e integrarlos utilizando técnicas de integración orientadas a servicios.

Nivel 2: Capa de componentes

Esta es la capa de los componentes de la organización que son responsables de implementar la funcionalidad y mantener la calidad de servicio de los servicios expuestos, así como los acuerdos a nivel de servicio de la aplicación en su conjunto. En este nivel típicamente podemos ubicar los servidores de aplicaciones y cualesquiera otras plataformas de ejecución de componentes software que ofrecen alta disponibilidad, balanceo de carga, tolerancia a fallos, etc.

Nivel 3: Capa de Servicios

Los servicios que se exponen para la construcción de procesos de negocio se exponen en este nivel, pudiendo ser descubiertos o directamente invocados, así como compuestos ya sea por orquestación o coreografía o bien utilizados individualmente.

A este nivel también se ubica la infraestructura de integración que permite la integración de servicios a través de la utilización de un conjunto de capacidades como son el enrutado de mensajes, mecanismos de transformación de información, protocolos de mediación, garantía de SLAs, etc., que son ofrecidos por infraestructuras como los ESB, de los que hablaremos en el próximo apartado.

Es a este nivel donde se exponen en algún mecanismo de registro (UDDI, ebXML RS) las descripciones de los servicios (mediante WSDL o el lenguaje de definición de servicios

considerado), que indicarán donde se localiza el servicio proporcionado, o bien mediante un ESB se proporcionará un mecanismo de integración independiente de la localización.

Nivel 4: Composición de procesos de negocio

Las composiciones y coreografías de servicios expuestos en el anterior nivel se definen en este, de forma que los servicios se agregan a un flujo de ejecución mediante un mecanismo de orquestación o coreografía, de forma que conforman una única aplicación.

Es en este nivel donde se ubica la infraestructura de BPM (y tienen aplicación los estándares BPML, XPD, WS-CDL, etc.), infraestructura que permite la definición de casos de uso y procesos de negocio, el uso de herramientas de simulación, así como el motor de ejecución que controla el flujo de tareas y actividades.

Nivel 5: Capa de acceso o presentación

Esta capa queda fuera del ámbito de las arquitecturas SOA, pero se ha convertido gradualmente en un nivel cada vez más relevante.

El motivo de su creciente importancia es la convergencia de estándares. Por un lado, hacia el mundo Web Service con el estándar WSRP (*Web Services for Remote Portlets*), que potencia el uso de servicios web para la interfaz de la aplicación. Por otro lado, la tendencia hacia el desacople del nivel de presentación, con infraestructuras de multicanalidad y multimodalidad que permiten abstraer la interfaz de usuario final al tiempo que convergen los medios, proporcionando independencia de la composición de servicios y construcción de procesos de negocio del canal de acceso que utilizará el usuario para su consumo.

Nivel 6: Calidad de Servicio, Seguridad y Gestión

Esta capa de carácter horizontal proporciona las capacidades necesarias para monitorizar, gestionar y mantener la calidad de servicio en términos de seguridad, transaccionalidad, prestaciones y disponibilidad.

Para ello, existirán procesos en segundo plano que actúan de sensores y herramientas que monitorizan la salud de las aplicaciones SOA, incluyendo entre ellas las implementaciones de estándares de gestión (como WS-Management o WS-DistributedManagement) y otros protocolos que implementan calidad de servicio en arquitecturas SOA.

En este nivel deberán considerarse las diferentes alternativas de securización de los servicios (WS-Security, WS-Trust, SAML, XACML, XKMS, XML-Signature, XML-Encryption, Liberty ID-FF, ID-WSF, ID-SIS, WS-Federation, WS-I Basic Security Profile), así como la transaccionalidad de las invocaciones a los mismos desde el nivel 4 de procesos de negocio al nivel 3 de servicios, mediante los estándares correspondientes (WS-AtomicTransaction, WS-Coordination, WS-CAF).

Nivel 7: Gobernanza SOA

Por encima de los aspectos generales de gestión y control de la seguridad y la transaccionalidad, la Gobernanza SOA es el mecanismo que permite asegurar la correcta y adecuada relación entre los servicios y el resto de los elementos que conforman la arquitectura SOA, de forma que se asegure el cumplimiento de leyes, políticas, estándares y procedimientos con los que la organización opera.

Mediante la Gobernanza, quedan definidos:

- Los roles relativos a la toma de decisiones y las tareas de implementación y despliegue.
- Las tareas asignadas a cada uno de los perfiles implicados.
- El ciclo de vida de los servicios.
- El control de las versiones de los servicios que pueden utilizarse en el diseño de los procesos de negocio.
- Elementos y medidas de evaluación del proceso.
- La cartera de proyectos SOA desarrollados.

2.6.3 El concepto de Enterprise Service Bus

Al igual que hemos visto para el concepto de BPM, existe una “moda” y tendencia en el mercado a vincular el concepto de *Enterprise Service Bus* (ESB) ineludiblemente con las arquitecturas SOA, por lo que vamos a analizar un poco más en detalle este concepto y lo que significa.

Enterprise Service Bus (ESB) es el concepto de moda de infraestructura para la creación de arquitecturas SOA, que permite la integración de servicios y abordando problemas reservados típicamente a aplicaciones EAI.

Como ocurría cuando buscábamos una definición de SOA, hay numerosas definiciones del concepto **ESB** que podemos encontrar en la literatura:

- “Categoría de productos y/o tecnologías middleware, basados en estándares de servicios, que habilitan la creación de arquitecturas basadas en servicios.” – *Wikipedia*.
- “Conjunto de capacidades y competencias que ha de tener una infraestructura para permitir la construcción de arquitecturas basadas en servicios (SOA). Por tanto, habrá de soportar interacciones basadas en servicios, mensajes y eventos en entornos heterogéneos, y ofrecer niveles de servicio y manejabilidad apropiados” – *IBM developerWorks*.
- “Nuevo modelo de arquitectura que explota las capacidades de los servicios, el middleware de mensajería, el enrutamiento inteligente y la transformación de mensajes. Se trata de un eje de integración ligero y ubicuo a través del que se pueden comunicar servicios y componentes de aplicaciones” – *Gartner Group*.
- “Patrón middleware que unifica y conecta servicios, aplicaciones y recursos dentro de una organización. Es un framework en el que las capacidades de las aplicaciones de un negocio se publican para su reutilización por parte de otras aplicaciones dentro de la propia organización o, incluso, fuera de los límites de ésta” – *IBM WebSphere*.

A pesar de que en el sector muchas empresas hablan sobre ESB como un tipo de productos específicos o meramente como una tecnología vinculada a SOA, el concepto ESB ni es sólo un producto ni es sólo una tecnología, sino un nuevo concepto: es la **definición de una infraestructura de integración para la construcción de arquitecturas SOA**.

Básicamente, un ESB es capaz de proporcionar las mismas capacidades que una solución EAI tradicional, como son la conectividad, los adaptadores de aplicaciones, el enrutamiento de mensajes basado en reglas y/o transformación de datos, pero basado en una arquitectura diferente a los EAI.

Mientras que en las soluciones EAI tradicionales las capacidades están implementadas dentro de una pila monolítica que constituye el sistema y el *middleware* ofrece todas las capacidades de integración, los sistemas ESB están implementados siguiendo los conceptos SOA, de tal modo que las **capacidades de integración son proporcionadas por servicios desplegados dentro del mismo bus de comunicación**.

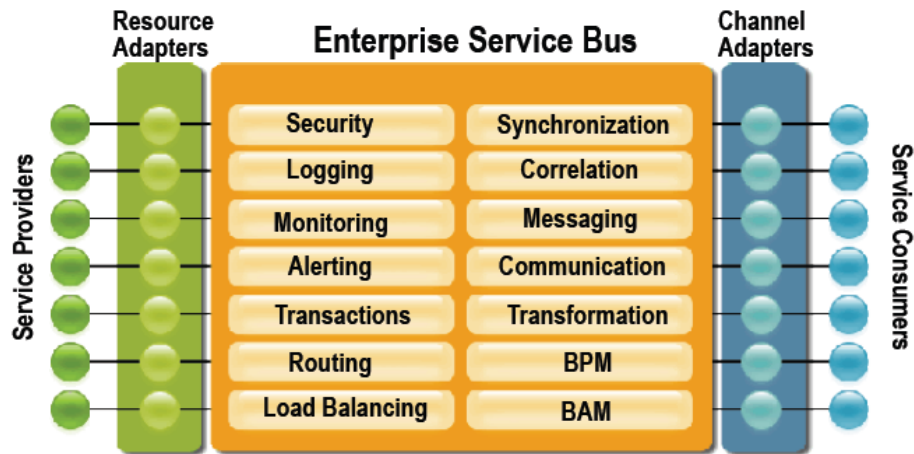


Ilustración 27. Capacidades de un ESB

Los productos comerciales disponibles en el mercado como soluciones ESB ofrecen gran cantidad de funciones, como servicios que refuerzan la seguridad, la transaccionalidad, sincronización, e incluso características de BPM, lo que ha motivado que SOA y ESB parezcan conceptos indisolubles.

No obstante, el conjunto mínimo de capacidades que ha de soportar un ESB es el siguiente:

1. A nivel de comunicación:
 - Servicios de enrutamiento y direccionamiento transparentes.
 - Al menos un protocolo de transporte de los de mayor aceptación.
 - Al menos un paradigma de mensajería.
2. A nivel de integración:
 - Soporte para múltiples medios de integración, como son Java 2 Connectors, Web Services, mensajería asíncrona, etc.
3. Otras:
 - Un modelo de interfaz abierto e independiente de la implementación que permita aislar/encapsular el código de las aplicaciones de los protocolos de transporte.

Junto a lo anterior, debe tenerse en cuenta que una solución basada en ESB, con independencia de otras funcionalidades que pueda ofrecer, debe de permitir:

- Que todas las aplicaciones integradas respondan de manera instantánea a nueva información.
- Minimizar los riesgos (p.ej. costes de evolución/mejora de sistemas) gracias a la utilización de protocolos de comunicación e interfaces estándares.
- Superar problemas como la diversidad de plataformas, arquitecturas y protocolos de red.
- Asegurar el correcto cumplimiento de los procesos transaccionales, incluso aunque algunos de los elementos integrantes no estén disponibles en la red.
- Enriquecer la información manejada por las aplicaciones sin tener que reprogramarlas.
- Disponer de una arquitectura altamente distribuida pero que puede ser gestionada de manera

centralizada.

- Distribuir información desde un negocio al exterior (p.ej. otras empresas o clientes).
- Creación incremental mediante la adición gradual de componentes, sin que ello suponga un problema para las aplicaciones integradas.
- Combinar tecnologías existentes con nuevos estándares y productos.
- Soportar arquitecturas orientadas tanto a servicio, como a mensajes y eventos.

Por consiguiente, un ESB es una solución óptima para ofrecer la integración de los servicios presentes en una arquitectura SOA, pero por supuesto, no es la única opción posible ni ineludiblemente el concepto de SOA tiene porqué tener asociado de forma directa el concepto de ESB.

3. Glosario

API: Application Program Interface.
BPEL: Business Process Execution Language.
BPM: Business Process Management.
BPMI: Business Process Management Initiative.
BPMN: Business Process Management Notation.
CORBA: Common Object Request Broker Architecture.
DCE: Distributed Computing Environment.
DCOM: Distributed Component Object Model.
DOM: Document Object Model.
EM: Estación de Mensajería.
ebXML: electronic business XML.
FTP: File Transfer Protocol.
HTTP: Hypertext Transfer Protocol.
IIOP: Internet Inter-ORB Protocol.
JMS: Java Message Service.
MOM: Message Oriented Middleware.
MSMQ: Microsoft Message Queuing.
NFS: Network File System.
NIS: Network Information Service.
OASIS: Organization for the Advancement of Structured Information Standards.
OMG: Object Management Group.
RMI: Remote Method Invocation.
RMI-IIOP: Remote Method Invocation – Internet Inter-ORB Protocol.
RPC: Remote Procedure Call.
SMTP: Simple Mail Transfer Protocol
SLA: Service Level Agreement.
SOA: Service Oriented Architecture.
SOAP: Simple Object Access Protocol.
SSI: Single System Image.
TLS: Transport Layer Security.
WMS: Workflow Management System.
WSDL: Webservices Description Language.
WS-I: Web Services Interoperability.
WSRP: Web Services for Remote Portlets.
XML: Extensible Markup Language.
XPDL: XML Process Definition Language.
XSL: Extensible Stylesheet Language.
XSL-FO: XSL Formatting Objects.
XSLT: XSL Transformations.

4. TEST

4.1 Preguntas de test

- 1) Señale cuál de las siguientes afirmaciones acerca de SOA es VERDADERA:
 - a. En la definición de una arquitectura de referencia SOA para una organización, una opción para cubrir las necesidades de comunicación de la misma es utilizar el estándar SOAP (SOA Protocol).
 - b. La implementación de una arquitectura SOA de referencia siempre se ha de realizar utilizando servicios web porque son la mejor opción para la implementación de una arquitectura SOA.
 - c. Entre los principios guía para el desarrollo de una arquitectura SOA se encuentran los principios de reutilización, interoperabilidad, composición y conformidad con estándares.
 - d. En el proceso de identificación de servicios de una organización se combinan las siguientes tres técnicas: enfoque bottom-up, enfoque middle-first y enfoque top-down.
- 2) En relación a las siguientes afirmaciones acerca de los sistemas distribuidos, señale la opción que NO es CIERTA:
 - a. Es un sistema distribuido, a la hora de coordinación y paso de mensajes, no existe un reloj común, por lo que se tienen que buscar formas alternativas de sincronización.
 - b. Los objetivos que buscan los sistemas distribuidos son: transparencia, fiabilidad, rendimiento, escalabilidad, flexibilidad y seguridad.
 - c. Los sistemas distribuidos son heterogéneos: tanto las arquitecturas de los nodos que componen el sistema distribuido, como los sistemas operativos de los nodos no tienen por qué ser iguales.
 - d. Los mecanismos que se suelen utilizar para la implementación de sistemas distribuidos son: paso de mensajes mediante las primitivas *send* y *receive*, *RPC* - llamada a procedimiento remoto y el uso de áreas de memoria compartida.
- 3) Indique cuál se las siguientes afirmaciones en relación con la arquitectura cliente-servidor NO es CORRECTA:
 - a. El modelo cliente-servidor es el modelo más simple y extendido de sistemas operativos distribuidos.
 - b. El modelo proxy/caché es un modelo de sistema distribuido evolucionado directamente del modelo cliente-servidor.
 - c. La utilización de un servidor proxy permite incrementar la disponibilidad, reducir la carga en las redes de comunicaciones y centralizar las peticiones de varios servicios en un único proxy.
 - d. En el modelo *Peer to Peer* existe una entidad central que asegura la adecuada comunicación entre los clientes y los servidores y proporciona servicios de monitorización y disponibilidad a los nodos que forman la red.

4.2 Solución a las preguntas de test

PREGUNTA	SOLUCIÓN
1	c
2	d
3	d

5. BIBLIOGRAFÍA

5.1 BIBLIOGRAFÍA BÁSICA

1. Service-Oriented Architecture - http://en.wikipedia.org/wiki/Service-oriented_architecture
2. Servicios Web XML - <http://www.w3.org/2002/ws/>
3. Arquitectura Servicios Web - <http://www.w3.org/TR/ws-arch/>
4. SOAP - <http://www.w3.org/TR/soap/>
5. WSDL - <http://www.w3.org/TR/wsdl>
6. UDDI - <http://uddi.xml.org/>
7. “SOA Reference Architecture - Defining The Key Elements Of A Successful SOA Technology Framework” – WebMethods

5.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. “Service-Oriented Architecture: Concepts, Technology, and Design” – Thomas Erl. Ed. Prentice Hall
2. SOA Practitioners’ Guide Part 1 Why Services-Oriented Architecture? – <http://www.soablueprint.com/whitepapers/SOAPGPart1.pdf>
3. SOA Practitioners’ Guide. Part 2. SOA Reference Architecture – <http://www.soablueprint.com/whitepapers/SOAPGPart2.pdf>
4. SOA Practitioners’ Guide. Part 3. Introduction to Services Lifecycle – <http://www.soablueprint.com/whitepapers/SOAPGPart3.pdf>
5. Patterns: Service-Oriented Architecture and Web Services - <https://www.redbooks.ibm.com/redbooks/pdfs/sg246303.pdf>